

On the Semantics of Generative SPARQL

Anonymous Author(s)

Abstract

We extend SPARQL with a generative query construct, called GenOp, whose evaluation calls a language model and produces typed solution mappings. We define the semantics of the GenOp in the query in a way that maintains the fixed-dataset assumption, on which formal semantics of SPARQL build, and extend solution mappings with values generated by the language model. We formalize the semantics of the extended language over these mappings using a compatibility relation that generalizes equality and supports similarity-based matching between RDF terms and generated values. We analyze the semantic consequences of generative query patterns, focusing on mapping-level recursion induced by the reuse of generated bindings. Under deterministic bounded generation and finite candidate coverage assumptions, we characterize acyclic and stratified fragments with fixpoint semantics, establish algebraic equivalences and semantics-preserving rewrite rules, and provide an executable evaluation method. With semantic exactness and treating generative-model calls and parsing as $O(1)$ per invocation, we further show that acyclic and stratified evaluation has PTIME data complexity and PSPACE-complete combined complexity; well-founded evaluation has PTIME data complexity and combined complexity that is PSPACE-hard and in EXPTIME.

ACM Reference Format:

Anonymous Author(s). 2026. On the Semantics of Generative SPARQL. In . ACM, New York, NY, USA, 25 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

SPARQL [33] is the de facto standard for querying RDF [27]. The semantics of SPARQL query patterns is defined in terms of (partial) solution mappings, with operators analogous to relational-algebra operators [8] and evaluated with respect to a fixed RDF graph [31, 32]. All semantic characterizations, equivalence results, and algebraic rewrite rules are stated relative to an underlying RDF graph that remains fixed during query evaluation [36].

Recent knowledge graph centric applications increasingly require *exploratory enrichment* over existing datasets [22], such as synthesizing missing attributes, proposing related entities, or producing textual descriptions. Large language models (LLMs) can generate such values on demand, but current practice treats calls to language models operationally, as external procedures, middleware invocations, or prompt-based post-processing, outside the query algebra [9, 24, 28, 35, 37, 42]. As a result, their interaction with

SPARQL operators is not governed by the declarative semantics of the query language, and standard questions about query equivalence, recursion, and termination remain outside the scope of these works.

One way to formally capture the semantics of operators that generate new values at query time is to relax the fixed-dataset assumption and allow the queried graph to be discovered during evaluation, as in link-traversal querying [19]. However, such approaches rely on dynamic data discovery and require specialized semantics that preclude algebraic equivalence, recursion, and fixpoint reasoning under a fixed graph [14, 19, 21]. In contrast, we aim to preserve the fixed-dataset assumption of SPARQL while *extending only the space of solution mappings* under a bounded generation assumption.

We integrate generative inference into the SPARQL algebra through a generative query pattern GenOp. A GenOp instantiates a prompt template using the currently bound variables, invokes a specified generative model, parses finitely many outputs, and returns them as bindings for designated output variables.

At the semantic level, our central design choice is to lift generation to the level of solution mappings rather than RDF graphs. To support algebraic composition with standard SPARQL operators, we define query semantics over *typed solution mappings* that distinguish RDF-derived bindings from generated bindings. Each binding records its origin, keeping evaluation relative to a fixed RDF graph. To combine native and generated values, whose textual realizations may differ despite semantic relatedness, we replace equality-based compatibility with a generalized compatibility relation: it coincides with equality on RDF bindings and allows similarity-based matching between RDF terms and generated values, following the general spirit of similarity-based querying [13, 25, 43], via deterministic lexicalization and a variable-indexed similarity threshold.

Although GenOp is non-recursive syntactically, cyclic dependencies may arise when generated bindings are reused as inputs to other GenOps. Such cycles occur solely at the level of extended solution mappings and induce self-referential constraints on admissible solutions. We formalize these dependencies via a consequence operator over sets of typed mappings and interpret cyclic queries using fixpoint semantics. We restrict ourselves to the deterministic bounded-generation assumption, where each fixed model specifier and instantiated prompt returns a fixed finite parsed output set, yielding a finite generated-value universe for each query instance. Under this assumption, query evaluation reduces to fixpoint computation over a finite mapping domain and therefore terminates by standard fixpoint arguments for finite lattices [1, 17, 38].

Let us explore the difficulty of including value-generating LLM calls into SPARQL query processing with the following example.

Example 1. Let $\mathcal{G} = \{:\text{Paris a :City}\}$ be an RDF graph, and consider the following pseudo-generative SPARQL query:

```
SELECT ?x ?y ?z WHERE {  
  ?x a :City .  
  GenOp("Suggest a topic related to ?x and ?z" AS ?y,
```

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

```

117                                     GPT-4o).
118   GenOp("Suggest a city related to topic ?y" AS ?z,
119         Gemini 1.5 Pro).}

```

Ideally, the first GenOp would behave analogously to a basic graph pattern: it should use available bindings and otherwise range over admissible combinations of variable assignments. Typical procedural implementations (e.g., [14]) require all input variables of a GenOp to be bound before invocation. In the given query, however, the first GenOp requires $?z$, produced by the second, while the second GenOp requires $?y$, produced by the first. Thus, no procedural evaluation order can invoke either operator first.

Our semantics interprets the resulting cyclic dependency declaratively over sets of typed solution mappings. Variables inside prompts are placeholders replaced by lexicalizations of their bindings, and the basic graph pattern “ $?x \text{ a :City}$ ” yields the initial mapping $\mu_0 = \{?x \mapsto \text{:Paris}\}$. Each GenOp induces a mapping transformer, and an extension $\mu \supseteq \mu_0$ assigning $?y$ and $?z$ is jointly self-supporting iff $?y$ is generated by the first GenOp from $\mu(?x)$ and $\mu(?z)$, while $?z$ is generated by the second GenOp from $\mu(?y)$. Such mutually supported mappings are characterized by the relevant fixpoint semantics.

For example, if the second GenOp generates $?z = \text{“Florence”}$ from $?y = \text{“art”}$, and the first, instantiated with $?x = \text{:Paris}$ and $?z = \text{“Florence”}$, generates $?y = \text{“art”}$, then the resulting extension of μ_0 is jointly self-supporting. The executable evaluation realizes this semantics by validating finite candidate mappings against these recursive conditions under the candidate-coverage assumptions in Section 6.

Our contributions are as follows. (i) We extend SPARQL with a generative operator GenOp and define a denotational semantics over typed solution mappings together with a generalized compatibility relation. Generated values remain confined to solution mappings and do not induce fact derivation, distinguishing the semantics from existential-rule languages and chase-based formalisms [1, 5, 6]. *Recursion is interpreted over mappings* rather than by model construction, distinguishing our approach from answer-set and stable-model approaches [29, 34]. (ii) We characterize acyclic, stratified, and cyclic generative patterns and provide fixpoint semantics for the recursive cases. Recursive admissibility is defined by a self-support condition. The underlying fixpoint theory is classical [1, 38], while its application to *context-dependent*, deterministic set-valued mapping transformers over solution mappings is *new*. (iii) We establish algebraic equivalence results and semantics-preserving rewrite rules for monotone and stratified fragments with respect to the fixpoint semantics over typed solution mappings, identifying both the preserved standard SPARQL algebraic laws and *new algebraic rewritings* in the presence of context-dependent generative patterns. (iv) We provide an executable realization of the recursive semantics by reducing evaluation to self-support checking over finite candidate mapping sets localized to strongly connected components of the dependency graph. This evaluation avoids chase-based reasoning and yields soundness and completeness relative to candidate coverage, following established treatments of logic programs with external operators [12]. In addition in Sect. 7, under bounded generation and semantic exactness, we show that acyclic and stratified evaluation matches standard SPARQL

complexity—PTIME in data complexity and PSPACE-complete in combined complexity—while well-founded evaluation is PTIME in data complexity and PSPACE-hard and in EXPTIME in combined complexity.

2 Related Work

The formal semantics of SPARQL as an algebra over solution mappings, together with results on containment, entailment and optimization under a fixed RDF dataset, are well established [3, 7, 23, 26, 32]. Our work builds on this foundation but departs from it by allowing solution mappings to be extended by generative operators during evaluation.

A distinct line of SPARQL extension for the Web of Linked Data has formalized semantics and computability under full-Web and reachability-based interpretations [18, 19], examined the empirical behavior and limitations of link-traversal query execution over heterogeneous Web data [39], and proposed dedicated Web-of-Data query languages that separate navigation from query evaluation, such as LDQL [20]. These approaches address open-world data discovery and computability over potentially unbounded graphs.

Recent systems integrate LLMs into knowledge-graph querying pipelines. SparqLLM invokes LLMs as auxiliary services during SPARQL execution [28], while other work uses declarative orchestration of LLM-assisted API and database calls around existing query engines [24]. Related approaches combine retrieval and generation in pipeline architectures [41]. In all cases, language model invocation remains external to the query algebra and is treated operationally, with generated outputs consumed procedurally rather than defined by the query semantics.

Related relational data-management work studies SQL and SQL-adjacent query processing with language-model calls, including hybrid relational-LLM querying, SQL-style querying of LLMs, and query engines over unstructured data [9, 35, 37, 42]. These works establish model invocation as a useful operational primitive in query processing and study how such calls can be exposed through SQL-style syntax, user-defined functions, physical operators, or dedicated execution engines. In all cases, focus is primarily execution-oriented and SQL-centric: model calls are incorporated as callable components of a query pipeline, without developing a formal denotational semantics for generated bindings inside a solution-mapping algebra.

Fixpoint semantics for recursive queries originate in deductive databases and logic programming [1, 38], with stratified and well-founded semantics providing principled treatments of negation [34, 40]. Existential rules and Datalog[±] extend these ideas via domain-expanding inference [5, 6]. Our setting differs fundamentally: the RDF domain remains fixed, no facts are derived, and recursion is resolved over a finite space of typed solution mappings. Treatments of logic programs with external operators [12] are conceptually closest, but does not address algebraic query languages or mapping-level recursion under fixed-dataset semantics.

3 Syntax and Semantics

We now introduce the syntax of generative SPARQL and define its denotational semantics in terms of typed solution mappings.

Let $\mathcal{V}$, $\mathcal{I}$, $\mathcal{B}$ and $\mathcal{L}$ be countably infinite disjoint sets of *variables*, *Internationalized Resource Identifiers* (IRIs), *blank nodes* and *literals*, respectively. Let $\mathcal{T} = \mathcal{I} \cup \mathcal{B} \cup \mathcal{L}$ be a set of RDF terms, and let $\mathcal{T}_{\text{gen}}$ be a disjoint set of *generated terms*. An *RDF triple* is an element of $(\mathcal{I} \cup \mathcal{B}) \times \mathcal{I} \times \mathcal{T}$, and a *triple pattern* is an element of $(\mathcal{I} \cup \mathcal{L} \cup \mathcal{V}) \times (\mathcal{I} \cup \mathcal{V}) \times (\mathcal{I} \cup \mathcal{L} \cup \mathcal{V})$. An *RDF graph* $\mathcal{G}$ is a finite set of RDF triples, and a *basic graph pattern* (BGP) is a finite set of triple patterns.

A generative SPARQL query is a *graph pattern* $\mathcal{P}$ defined by the following grammar:

$$\begin{aligned} \mathcal{P} := & \text{BGP} \mid \text{GenOp}(\pi(X), Y, \mathcal{M}) \mid \text{Filter}_{\mathcal{F}}(\mathcal{P}) \\ & \mid \text{Proj}_L(\mathcal{P}) \mid \mathcal{P} \text{ Union } \mathcal{P} \mid \mathcal{P} \text{ Join } \mathcal{P} \\ & \mid \mathcal{P} \text{ Minus } \mathcal{P} \mid \mathcal{P} \text{ Diff }_{\mathcal{F}} \mathcal{P} \mid \mathcal{P} \text{ Opt }_{\mathcal{F}} \mathcal{P}, \end{aligned} \quad (1)$$

where $\text{GenOp}(\pi(X), Y, \mathcal{M})$ is a *generative pattern* with prompt template π containing placeholders $X \subseteq \mathcal{V}$, an ordered tuple $Y = \langle y_1, \dots, y_k \rangle$ of distinct output variables from $\mathcal{V}$ with $X \cap Y = \emptyset$ (set-theoretic notation on Y refers to its underlying set), and generative model specifier $\mathcal{M}$ (abstract); $L \subseteq \mathcal{V}$ is a projection list, and $\mathcal{F}$ is a filter formula built from atoms $\text{bound}(v)$, $(v = c)$, $(v = v')$ s.t. $v, v' \in \mathcal{V}$ and $c \in \mathcal{T}$ using logical connectives $\wedge$ and $\neg$. The set of variables in a graph pattern $\mathcal{P}$ is denoted by $\text{var}(\mathcal{P})$, and the size of $\mathcal{P}$, i.e., the number of symbols in its writing, by $|\mathcal{P}|$.

Definition 1. Let $\mathcal{P}$ be a graph pattern. A BGP or generative GenOp pattern Q is said to occur in $\mathcal{P}$, written $Q \preceq \mathcal{P}$, if Q is a syntactic subpattern of $\mathcal{P}$, defined inductively by: (i) if $\mathcal{P} = Q$, then $Q \preceq \mathcal{P}$; (ii) if $\mathcal{P} = \text{Filter}_{\mathcal{F}}(\mathcal{P}')$ or $\mathcal{P} = \text{Proj}_L(\mathcal{P}')$, then $Q \preceq \mathcal{P}$ iff $Q \preceq \mathcal{P}'$; (iii) if $\mathcal{P} = \mathcal{P}_1 \text{ Op } \mathcal{P}_2$ for $\text{Op} \in \{\text{Join}, \text{Union}, \text{Minus}, \text{Diff}, \text{Opt}\}$, then $Q \preceq \mathcal{P}$ iff $Q \preceq \mathcal{P}_1$ or $Q \preceq \mathcal{P}_2$.

Let $\{\}$ be the empty graph pattern, with $\llbracket \{\} \rrbracket_{\mathcal{G}} = \{\emptyset\}$ and $\text{var}(\{\}) = \emptyset$.

Definition 2. Let $\mathcal{P}$ be a graph pattern and let $Q \preceq \mathcal{P}$ be a designated occurrence. The *surrounding pattern* of Q in $\mathcal{P}$, denoted $\mathcal{P} \setminus \{Q\}$, is obtained by replacing only that occurrence by the empty pattern $\{\}$ and then simplifying the result by the following pruning $\rightsquigarrow$ rules:

- (1) $\text{Filter}_{\mathcal{F}}(\{\}) \rightsquigarrow \{\}$ and $\text{Proj}_L(\{\}) \rightsquigarrow \{\}$;
- (2) For the $\text{Op} \in \{\text{Join}, \text{Union}, \text{Minus}, \text{Diff}, \text{Opt}\}$ and graph pattern $\mathcal{P}'$, $\{\} \text{ Op } \mathcal{P}' \rightsquigarrow \mathcal{P}'$ and $\mathcal{P}' \text{ Op } \{\} \rightsquigarrow \mathcal{P}'$;

provided that the resulting graph pattern is *well formed*.

Definition 3. (Well formed) A graph pattern $\mathcal{P}$ defined by the grammar 1 is *well-formed* if it satisfies the following conditions: (i) for every $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$ occurrence such that $g \preceq \mathcal{P}$, $X \subseteq \text{var}(\mathcal{P} \setminus \{g\}) \setminus Y$; (ii) for every filter $\text{Filter}_{\mathcal{F}}(\mathcal{P})$, $\text{var}(\mathcal{F}) \subseteq \text{var}(\mathcal{P})$; and (iii) for every projection $\text{Proj}_L(\mathcal{P})$, $L \subseteq \text{var}(\mathcal{P})$.

We now define solution mappings, typing, and compatibility.

Definition 4. A *typed term* is a pair (t, τ) where either (i) $t \in \mathcal{T}$ and $\tau = \text{rdf}$, or (ii) $t \in \mathcal{T}_{\text{gen}}$ and $\tau = \text{gen}$. A *typed solution mapping* is a partial function $\mu : \mathcal{V} \rightarrow (\mathcal{T} \times \{\text{rdf}\}) \cup (\mathcal{T}_{\text{gen}} \times \{\text{gen}\})$ with domain $\text{dom}_{\mu} \subseteq \mathcal{V}$.

Let Σ be a finite alphabet and Σ^* be the set of all finite strings over Σ .

Definition 5. A *canonical lexicalization* consists of fixed deterministic functions

$$\text{Canon} : \mathcal{T} \rightarrow \Sigma^* \quad \text{and} \quad \text{Lex} : \mathcal{T}_{\text{gen}} \rightarrow \Sigma^*,$$

where Canon maps each RDF term to a canonical string and Lex maps each generated term to its textual realization.

Remark 1. In an implementation, literals can be lexicalized by their lexical forms. IRIs can be lexicalized by a deterministic label policy, for example by using an associated *rdfs:label* when available and otherwise a stable rendering of the IRI. Blank nodes can be lexicalized by a stable system-defined identifier or by a deterministic bounded-neighbourhood description. We require only that the lexicalization policy is fixed during evaluation; the semantics does not require labels or blank-node descriptions to be semantically complete.

Definition 6. Let $\text{Str}_{\approx} : \Sigma^* \times \Sigma^* \rightarrow [0, 1]$ be a symmetric similarity function, and let $\theta : \mathcal{V} \rightarrow (0, 1]$ be a variable-specific *threshold function* (default 1). For any two typed terms (t, τ) , (t', τ') , we define Sim as,

$$\text{Sim}((t, \tau), (t', \tau')) = \begin{cases} 1 & \text{if } (\tau = \tau' = \text{rdf}) \wedge (t = t') \\ 0 & \text{if } (\tau = \tau' = \text{rdf}) \wedge (t \neq t') \\ \text{Str}_{\approx}(\text{Canon}(t), \text{Lex}(t')) & \text{if } \tau = \text{rdf}, \tau' = \text{gen} \\ \text{Str}_{\approx}(\text{Lex}(t), \text{Canon}(t')) & \text{if } \tau = \text{gen}, \tau' = \text{rdf} \\ \text{Str}_{\approx}(\text{Lex}(t), \text{Lex}(t')) & \text{if } \tau = \tau' = \text{gen} \end{cases}$$

Definition 7 (Variable-indexed similarity). For each variable $v \in \mathcal{V}$, let $\approx_v$ be the binary relation on typed terms defined by

$$\alpha \approx_v \beta \quad \text{iff} \quad \text{Sim}(\alpha, \beta) \geq \theta(v).$$

Since only finitely many typed terms can bind a given variable under the bounded-generation assumption, the reflexive and transitive closure of $\approx_v$ therefore induces finitely many equivalence classes; we fix a deterministic total order $\preceq$ on typed terms, and write $\text{rep}_v(\alpha)$ for the $\preceq$ -minimal representative of the equivalence class of α under $\approx_v$.

Definition 8. For a variable v and typed terms α, β , we define *canonical compatibility*,

$$\text{Compat}_v(\alpha, \beta) = \top \quad \text{iff} \quad \text{rep}_v(\alpha) = \text{rep}_v(\beta).$$

Definition 9. Two typed solution mappings μ_1 and μ_2 are *compatible*, written $\mu_1 \sim \mu_2$, iff for every variable $v \in \text{dom}_{\mu_1} \cap \text{dom}_{\mu_2}$,

$$\text{Compat}_v(\mu_1(v), \mu_2(v)) = \top.$$

If $\mu_1 \sim \mu_2$, their *canonical union* $\mu_1 \uplus \mu_2$ is the mapping with domain $\text{dom}_{\mu_1} \cup \text{dom}_{\mu_2}$ defined by

$$(\mu_1 \uplus \mu_2)(v) = \begin{cases} \text{rep}_v(\mu_1(v)) & \text{if } v \in \text{dom}_{\mu_1} \cap \text{dom}_{\mu_2}, \\ \mu_1(v) & \text{if } v \in \text{dom}_{\mu_1} \setminus \text{dom}_{\mu_2}, \\ \mu_2(v) & \text{if } v \in \text{dom}_{\mu_2} \setminus \text{dom}_{\mu_1}. \end{cases}$$

If $\mu_1 \not\sim \mu_2$, the union $\mu_1 \uplus \mu_2$ is undefined.

By slight abuse of notation, for a graph pattern $\mathcal{P}$ and mapping μ , we write $\mu(\mathcal{P})$ for the set of triples obtained by substituting the variables $\text{var}(\mathcal{P})$ in $\mathcal{P}$ according to μ .

BGP evaluation. Let $\mathcal{P}$ be a BGP and $\mathcal{G}$ an RDF graph. The evaluation of $\mathcal{P}$ over $\mathcal{G}$ is the set,

$$\llbracket \mathcal{P} \rrbracket_{\mathcal{G}} = \{ \mu \mid \text{dom}_{\mu} = \text{var}(\mathcal{P}) \wedge \mu(\mathcal{P}) \subseteq \mathcal{G} \},$$

where each binding $\mu(v) = (t, \text{rdf})$ with $t \in \mathcal{T}$.

3.1 Generative patterns

We model a generative pattern as a set-valued mapping that transforms an instantiated prompt into finitely many tuples of generated terms via a generative step followed by parsing, while leaving the surrounding query algebra unchanged.

A model specifier $\mathcal{M}$ induces finite-valued operations $\text{Call}_{\mathcal{M}} : \Sigma^* \rightarrow 2^{\Sigma^*}$ and $\text{Parse}_{\mathcal{M}}$, where $\text{Call}_{\mathcal{M}}$ returns answer strings and $\text{Parse}_{\mathcal{M}}$ returns tuples of generated terms. For every fixed query instance $(\mathcal{P}, \mathcal{G})$, we assume that iterating the generative occurrences of $\mathcal{P}$ from the RDF bindings in $\mathcal{G}$ yields finitely many generated terms. For complexity results, we assume constants n_1, n_2 such that

$$|\text{Call}_{\mathcal{M}}(s)| \leq n_1 \quad \text{and} \quad |\text{Parse}_{\mathcal{M}}(a)| \leq n_2$$

for all relevant $\mathcal{M}, s, a$, and that generated terms have bounded representation length.

Definition 10. Let $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$, where $Y = \langle y_1, \dots, y_k \rangle$ is the ordered output tuple and $k = |Y|$. Let $\text{Call}_{\mathcal{M}} : \Sigma^* \rightarrow 2^{\Sigma^*}$ and $\text{Parse}_{\mathcal{M}} : \Sigma^* \rightarrow 2^{\mathcal{T}_{\text{gen}}^{|Y|}}$ return finite sets. Let $\mathcal{G}$ be an RDF graph and let S be a context supply, i.e., a set of typed solution mappings, such that $X \subseteq \text{dom}_{\mu}$ for every $\mu \in S$. The evaluation of g with respect to S over $\mathcal{G}$ is:

$$\llbracket g \rrbracket_{\mathcal{G}}^S = \bigcup_{\mu \in S} \left\{ \mu \uplus \nu \mid \begin{array}{l} \mathbf{a} \in \text{Call}_{\mathcal{M}}(\pi^{[\mu]}), \\ (v_1, \dots, v_k) \in \text{Parse}_{\mathcal{M}}(\mathbf{a}), \\ \nu = \{y_i \mapsto (v_i, \text{gen}) \mid 1 \leq i \leq k\}, \\ \mu \sim \nu, \end{array} \right\}$$

where $\pi^{[\mu]}$ is the prompt obtained from π by replacing each placeholder in X by the lexicalization of its binding under μ .

3.2 Filter

Let μ be a typed solution mapping. The truth value of a filter formula $\mathcal{F}$ under μ , denoted $\mathcal{F}^{\mu} \in \{\top, \perp, \varepsilon\}$ (where ε denotes an error), is defined inductively as follows.

- (1) $\text{bound}(v)^{\mu} = \top$ if $v \in \text{dom}_{\mu}$, and $\text{bound}(v)^{\mu} = \perp$ otherwise.
- (2) For $c \in \mathcal{T}$, $(v = c)^{\mu} = \varepsilon$ if $v \notin \text{dom}_{\mu}$; $(v = c)^{\mu} = \top$ if $\mu(v) = (c, \text{rdf})$ or $\text{Compat}_v(\mu(v), (c, \text{rdf})) = \top$; otherwise, $(v = c)^{\mu} = \perp$.
- (3) $(v = v')^{\mu} = \varepsilon$ if $\{v, v'\} \not\subseteq \text{dom}_{\mu}$. Otherwise, $(v = v')^{\mu} = \top$ if either (i) $\mu(v) = \mu(v') = (t, \text{rdf})$ for some $t \in \mathcal{T}$ or (ii) $\text{Compat}_v(\mu(v), \mu(v')) = \top$; otherwise, $(v = v')^{\mu} = \perp$.
- (4) $(\neg \mathcal{F})^{\mu} = \top$ if $\mathcal{F}^{\mu} = \perp$; $(\neg \mathcal{F})^{\mu} = \perp$ if $\mathcal{F}^{\mu} = \top$; $(\neg \mathcal{F})^{\mu} = \varepsilon$ if $\mathcal{F}^{\mu} = \varepsilon$.
- (5) $(\mathcal{F}_1 \wedge \mathcal{F}_2)^{\mu} = \mathcal{F}_1^{\mu} \wedge \mathcal{F}_2^{\mu}$.

3.3 Graph Patterns

The evaluation of a graph pattern $\mathcal{P}$ over an RDF graph $\mathcal{G}$, denoted $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S$, returns a set of typed solution mappings and is defined recursively below relative to a context supply S . When the context supply is fixed or immaterial, we write $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}$ as shorthand for $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S$. The clauses below define a *single-pass* (one-step) evaluation; cyclic

generative dependencies are handled by the fixpoint semantics in Section 4.

- (1) If $\mathcal{P}$ is a BGP, then $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}$ is defined as above. If $\mathcal{P}$ is a GenOp pattern, its single-pass evaluation is $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S$, where S provides the solution mappings used to instantiate the placeholder variables of its prompt template.
- (2) $\llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P}) \rrbracket_{\mathcal{G}} = \{ \mu \in \llbracket \mathcal{P} \rrbracket_{\mathcal{G}} \mid \mathcal{F}^{\mu} = \top \}$;
- (3) $\llbracket \text{Proj}_L(\mathcal{P}) \rrbracket_{\mathcal{G}} = \{ \mu|_L \mid \mu \in \llbracket \mathcal{P} \rrbracket_{\mathcal{G}} \}$;
- (4) $\llbracket \mathcal{P}_1 \text{ Join } \mathcal{P}_2 \rrbracket_{\mathcal{G}} = \{ \mu_1 \uplus \mu_2 \mid \mu_1 \in \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}}, \mu_2 \in \llbracket \mathcal{P}_2 \rrbracket_{\mathcal{G}}, \mu_1 \sim \mu_2 \}$;
- (5) $\llbracket \mathcal{P}_1 \text{ Union } \mathcal{P}_2 \rrbracket_{\mathcal{G}} = \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}} \cup \llbracket \mathcal{P}_2 \rrbracket_{\mathcal{G}}$;
- (6) $\llbracket \mathcal{P}_1 \text{ Diff }_{\mathcal{F}} \mathcal{P}_2 \rrbracket_{\mathcal{G}} = \{ \mu_1 \in \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}} \mid \forall \mu_2 \in \llbracket \mathcal{P}_2 \rrbracket_{\mathcal{G}} : \mu_1 \sim \mu_2 \text{ or } \mathcal{F}^{\mu_1 \uplus \mu_2} \neq \top \}$;
- (7) $\llbracket \mathcal{P}_1 \text{ Opt }_{\mathcal{F}} \mathcal{P}_2 \rrbracket_{\mathcal{G}} = \llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P}_1 \text{ Join } \mathcal{P}_2) \rrbracket_{\mathcal{G}} \cup \llbracket \mathcal{P}_1 \text{ Diff }_{\mathcal{F}} \mathcal{P}_2 \rrbracket_{\mathcal{G}}$;
- (8) $\llbracket \mathcal{P}_1 \text{ Minus } \mathcal{P}_2 \rrbracket_{\mathcal{G}} = \{ \mu_1 \in \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}} \mid \forall \mu_2 \in \llbracket \mathcal{P}_2 \rrbracket_{\mathcal{G}} : \text{either } \mu_1 \sim \mu_2 \text{ or } \text{dom}_{\mu_1} \cap \text{dom}_{\mu_2} = \emptyset \}$;

where $\mu|_L$ is the restriction of μ to L . For a context supply S' , i.e., a set of typed solution mappings, and $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$, write $S'|_g = \{ \mu \in S' \mid X \subseteq \text{dom}_{\mu} \}$. If $g \leq \mathcal{P}$, its context supply in a single evaluation step under supply S is

$$S_g = \begin{cases} \{\emptyset\}, & X = \emptyset, \\ \llbracket \mathcal{P} \setminus \{g\} \rrbracket_{\mathcal{G}}^S|_g, & X \neq \emptyset. \end{cases}$$

Proposition 1. Let $\mathcal{P}$ be a well-formed graph pattern that contains no occurrence of GenOp. Then, for any RDF graph $\mathcal{G}$, the evaluation of $\mathcal{P}$ under the extended semantics coincides with standard SPARQL evaluation under set semantics: $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}} = \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^{\text{standard SPARQL}}$.

4 Fragments and Fixpoint Semantics

The preceding semantics characterize the meaning of generative SPARQL queries independently of any evaluation strategy. Section 6 presents an executable realization of these semantics under bounded generation.

4.1 Dependency Graph

Let $\mathcal{P}$ be a generative SPARQL graph pattern and let $\text{gen}(\mathcal{P}) = \{g \mid g \leq \mathcal{P} \text{ and } g \text{ is a GenOp occurrence}\}$ denote the set of GenOp occurrences in $\mathcal{P}$. For each $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$ in $\text{gen}(\mathcal{P})$, let $\text{input}(g)$ denote its set of placeholder variables X and $\text{out}(g)$ its set of output variables Y .

Definition 11 (Dependency graph). The *dependency graph* of $\mathcal{P}$ is the directed graph $\vec{\mathcal{D}}_{\mathcal{P}} = (\text{gen}(\mathcal{P}), \rightarrow_{\mathcal{P}})$, where for $g_i, g_j \in \text{gen}(\mathcal{P})$,

$$g_i \rightarrow_{\mathcal{P}} g_j \quad \text{iff} \quad \text{out}(g_i) \cap \text{input}(g_j) \neq \emptyset.$$

Definition 12. A nonempty subset $C \subseteq \text{gen}(\mathcal{P})$ is a *strongly connected component* (SCC) of $\mathcal{P}$ iff:

- (1) for all $g, g' \in C$, there exists a finite sequence $g = g_0, \dots, g_k = g'$ in C such that $g_i \rightarrow_{\mathcal{P}} g_{i+1}$ for all $i < k$, and a sequence from g' to g ; and
- (2) C is maximal w.r.t. set inclusion under this property.

The SCCs of $\mathcal{P}$ form a partition of $\text{gen}(\mathcal{P})$.

Definition 13 (Acyclicity). A graph pattern $\mathcal{P}$ is *acyclic* iff $\vec{\mathcal{D}}_{\mathcal{P}}$ is acyclic.

If $\mathcal{P}$ is acyclic and monotone with respect to the context supply, all GenOp occurrences can be evaluated in a single pass following a topological order of $\vec{\mathcal{D}}_{\mathcal{P}}$, and the staged result equals $\text{lfp}(\Phi_{\mathcal{P}})$ (Cf. Section 4.3).

Definition 14. (Signed Dependency Edge) Let $g_i \rightarrow_{\mathcal{P}} g_j$ be a dependency edge and let $V_{ij} = \text{out}(g_i) \cap \text{input}(g_j)$. Let $g_j \preceq \mathcal{P}$. We classify the edge by the syntactic occurrences of variable $v \in V_{ij}$ in the surrounding pattern $\mathcal{P} \setminus \{g_j\}$.

- (1) The edge is *positive* if every syntactic occurrence of every $v \in V_{ij}$ in $\mathcal{P} \setminus \{g_j\}$ is in a *positive position*, namely under only the operators Join, Union, Proj, the Join branch of Opt or within a Filter where v does not occur under $\neg$.
- (2) The edge is *negative* if there exists some $v \in V_{ij}$ that has a syntactic occurrence in $\mathcal{P} \setminus \{g_j\}$ in a *negative position*, i.e., within the scope of Minus, Diff, the Diff branch of Opt or under $\neg$ in a Filter.

Example 2. Let g_1 and g_2 be the two GenOp occurrences in

```
SELECT ?x ?y ?z WHERE {
  ?x a :City .
  GenOp("Find topic for ?x" AS ?y, GPT-4o).
  GenOp("Find city for ?y" AS ?z, GPT-4o). }
```

The variable $?y$ occurs as a placeholder in the prompt of g_2 and only in positive positions in the surrounding pattern of g_2 ; hence $g_1 \rightarrow g_2$ is a positive dependency. In the variant obtained by adding MINUS { ?x :hasTopic ?y . }

the variable $?y$ occurs under a negative operator in the surrounding pattern of g_2 . Thus, $g_1 \rightarrow g_2$ is a negative dependency, and any stratification as in Definition 15 must satisfy $\sigma(g_1) < \sigma(g_2)$.

Definition 15. A *stratification* of $\mathcal{P}$ is a function $\sigma : \text{gen}(\mathcal{P}) \rightarrow \mathbb{N}$ such that for every $g_i \rightarrow_{\mathcal{P}} g_j$: (i) if the edge is positive then $\sigma(g_i) \leq \sigma(g_j)$; (ii) if the edge is negative then $\sigma(g_i) < \sigma(g_j)$. The pattern $\mathcal{P}$ is *stratified* if such a function exists.

Acyclic patterns admit single-pass evaluation. Stratified patterns admit stratum-wise fixpoint semantics. Non-stratified patterns require well-founded semantics.

4.2 Consequence Operator

We handle recursion among GenOp patterns via an immediate consequence operator, assuming bounded generation so that, for fixed $\mathcal{P}$ and $\mathcal{G}$, the set of generated terms, and thus relevant typed solution mappings, is finite.

Let $\Omega_{\mathcal{P}}$ be the finite set of all typed solution mappings over $\text{var}(\mathcal{P})$ whose values range over the finite subset of $\mathcal{T} \cup \mathcal{T}_{\text{gen}}$. To expose recursive dependencies, evaluation is parameterized by a *context supply* $S \subseteq \Omega_{\mathcal{P}}$. For each occurrence $g \preceq \mathcal{P}$, we use its context supply S_g defined in Section 3; thus g is evaluated using $\llbracket g \rrbracket_{\mathcal{G}}^{S_g}$.

Definition 16. The immediate consequence operator of $\mathcal{P}$ is the function

$$\Phi_{\mathcal{P}} : 2^{\Omega_{\mathcal{P}}} \rightarrow 2^{\Omega_{\mathcal{P}}}, \quad \Phi_{\mathcal{P}}(S) := \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S.$$

The $\Phi_{\mathcal{P}}(S)$ evaluates the query once while allowing each GenOp occurrence g to draw its placeholder bindings from S_g . Recursive behavior is captured by iterating $\Phi_{\mathcal{P}}$ to a fixpoint.

Proposition 2. If $S, S' \subseteq \Omega_{\mathcal{P}}$ satisfy $S_g = S'_g$ for every GenOp occurrence g in $\mathcal{P}$, then $\Phi_{\mathcal{P}}(S) = \Phi_{\mathcal{P}}(S')$.

4.3 Monotonicity and Least Fixpoint Semantics

We identify conditions for a least-fixpoint semantics by establishing monotonicity of the consequence operator.

A generative SPARQL pattern $\mathcal{P}$ is monotone w.r.t. the context supply if, for every GenOp $g \preceq \mathcal{P}$, variables in $\text{out}(g)$ occur only in positive positions in $\mathcal{P}$, so added generated bindings cannot invalidate existing solution mappings.

Lemma 1. If $\mathcal{P}$ is monotone with respect to the context supply, then the consequence operator $\Phi_{\mathcal{P}}$ is monotone on the complete lattice $(2^{\Omega_{\mathcal{P}}}, \subseteq)$:

$$\forall S, S' \in 2^{\Omega_{\mathcal{P}}} : S \subseteq S' \longrightarrow \Phi_{\mathcal{P}}(S) \subseteq \Phi_{\mathcal{P}}(S').$$

If $\Phi_{\mathcal{P}}$ is monotone, it admits a least fixpoint with respect to set inclusion. Under the bounded-generation assumption, this fixpoint can be computed by iterating $\Phi_{\mathcal{P}}$ from the empty set.

Definition 17 (Least fixpoint semantics). Let $\Phi_{\mathcal{P}}$ be monotone, and the ascending sequence $\{S_n\}_{n \geq 0}$ is defined by

$$S_0 = \emptyset \quad \text{and} \quad S_{n+1} = \Phi_{\mathcal{P}}(S_n).$$

The *least fixpoint* of $\Phi_{\mathcal{P}}$ is $\text{lfp}(\Phi_{\mathcal{P}}) = \bigcup_{n \geq 0} S_n$, and the meaning of $\mathcal{P}$ under least-fixpoint semantics is $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^{\text{lfp}} := \text{lfp}(\Phi_{\mathcal{P}})$.

Theorem 1. (Existence and characterization) If $\Phi_{\mathcal{P}}$ is monotone on $(2^{\Omega_{\mathcal{P}}}, \subseteq)$, then it has a least fixpoint, and $\text{lfp}(\Phi_{\mathcal{P}}) = \bigcup_{n \geq 0} \Phi_{\mathcal{P}}^n(\emptyset)$. Moreover, if $\Omega_{\mathcal{P}}$ is finite, the sequence stabilizes after at most $|\Omega_{\mathcal{P}}|$ iterations.

For acyclic patterns that are monotone with respect to the context supply, topological staged evaluation yields $\text{lfp}(\Phi_{\mathcal{P}})$.

Theorem 2. Let $S^* = \text{lfp}(\Phi_{\mathcal{P}})$. If S is any set of typed solution mappings such that $\Phi_{\mathcal{P}}(S) \subseteq S$, then $S^* \subseteq S$. Thus, S^* is the minimal fixpoint model of $\mathcal{P}$.

4.4 Stratified Semantics

Let $\mathcal{P}$ be a generative SPARQL pattern and let $\sigma : \text{gen}(\mathcal{P}) \rightarrow \mathbb{N}$ be a stratification. For $i \in \mathbb{N}$, let

$$\text{Gen}^{\leq i}(\mathcal{P}) = \{g \in \text{gen}(\mathcal{P}) \mid \sigma(g) \leq i\}.$$

Let $\mathcal{P}^{\leq i}$ denote the pattern obtained from $\mathcal{P}$ by *disabling* all GenOp occurrences not in $\text{Gen}^{\leq i}(\mathcal{P})$, that is, replacing each such occurrence by a pattern whose evaluation is empty for all context supply S , while leaving the remaining algebraic structure unchanged.

Definition 18. (Stratum-wise operator) For each i , let $\Phi_{\mathcal{P}^{\leq i}} : 2^{\Omega_{\mathcal{P}}} \rightarrow 2^{\Omega_{\mathcal{P}}}$ defined by $\Phi_{\mathcal{P}^{\leq i}}(S) = \llbracket \mathcal{P}^{\leq i} \rrbracket_{\mathcal{G}}^S$.

Definition 19. (Stratified semantics) Let $\mathcal{P}$ be stratified with stratification $\sigma : \text{gen}(\mathcal{P}) \rightarrow \mathbb{N}$. Let $S_{-1} = \emptyset$. For each stratum index $i = 0, 1, 2, \dots$, consider the operator $S \mapsto \Phi_{\mathcal{P}^{\leq i}}(S \cup S_{i-1})$, and let S_i be its least fixpoint:

$$S_i = \text{lfp}(S \mapsto \Phi_{\mathcal{P}^{\leq i}}(S \cup S_{i-1})).$$

Let $m = \max\{\sigma(g) \mid g \in \text{gen}(\mathcal{P})\}$, with $m = 0$ if $\text{gen}(\mathcal{P}) = \emptyset$. The meaning of $\mathcal{P}$ under stratified semantics is given by $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^{\text{strat}} = S_m$.

Theorem 3. *If $\mathcal{P}$ is stratified and $\Omega_{\mathcal{P}}$ is finite (bounded generation), then the stratified semantics $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^{\text{strat}}$ is well-defined and unique.*

Theorem 4. *Let $\mathcal{P}$ be stratified. For each stratum i , the least fixpoint S_i is independent of the order in which GenOp occurrences in stratum i are evaluated.*

4.5 Well-Founded Semantics for Non-Stratified Patterns

For query patterns that are not stratified, we adopt the standard three-valued semantics based on an alternating fixpoint construction [34, 40], instantiated to typed solution mappings.

Let $\text{Cand}(\mathcal{P}) \subseteq \Omega_{\mathcal{P}}$ be a finite candidate set, assumed to be a sound over-approximation of all typed solution mappings that may arise during evaluation of $\mathcal{P}$ under bounded generation. A partial interpretation is a pair (I^+, I^-) with $I^+, I^- \subseteq \text{Cand}(\mathcal{P})$ and $I^+ \cap I^- = \emptyset$, where mappings in I^+ are true, mappings in I^- are false, and all others are undefined.

Definition 20. Let $\{\Phi_{\mathcal{P}}^J\}_{J \subseteq \text{Cand}(\mathcal{P})}$ be a reduct family, where each $\Phi_{\mathcal{P}}^J : 2^{\text{Cand}(\mathcal{P})} \rightarrow 2^{\text{Cand}(\mathcal{P})}$ is obtained by fixing the negative branches of MINUS, DIFF, and the DIFF branch of OPT against J ; pointwise filter negation remains as in Section 3. The reduct family satisfies, for all $S \subseteq S'$ and $J \subseteq J'$,

$$\Phi_{\mathcal{P}}^J(S) \subseteq \Phi_{\mathcal{P}}^J(S') \quad \text{and} \quad \Phi_{\mathcal{P}}^{J'}(S) \subseteq \Phi_{\mathcal{P}}^J(S).$$

These are the monotonicity-in- S and antitonicity-in- J conditions of the alternating-fixpoint reduct used below.

Definition 21. Let $(I_0^+, I_0^-) := (\emptyset, \emptyset)$. For $n \geq 0$,

- (1) $I_{n+1}^+ = \text{lfp}(S \mapsto \Phi_{\mathcal{P}}^{\text{Cand}(\mathcal{P}) \setminus I_n^-}(S))$,
- (2) $I_{n+1}^- = \text{Cand}(\mathcal{P}) \setminus \text{lfp}(S \mapsto \Phi_{\mathcal{P}}^{I_{n+1}^+}(S))$.

When the sequence stabilizes at (I^+, I^-) , we call this pair the well-founded model of $\mathcal{P}$ over $\mathcal{G}$ relative to $\text{Cand}(\mathcal{P})$ and the fixed reduct family.

Theorem 5. (Convergence and uniqueness) *If $\text{Cand}(\mathcal{P})$ is finite, then the alternating fixpoint sequence in Definition 21 stabilizes after finitely many steps at a pair (I^+, I^-) . The induced three-valued interpretation is unique; it is the well-founded model of $\mathcal{P}$ over $\mathcal{G}$ relative to $\text{Cand}(\mathcal{P})$ and the fixed reduct family.*

Non-stratified generative SPARQL patterns with cyclic negative dependencies may be non-monotone. Under bounded generation, they admit a three-valued well-founded semantics via the standard alternating-fixpoint construction over the finite mapping lattice.

5 Algebraic Equivalences and Rewrite Rules

We prove equivalence results for the monotone or stratified fragments, where denotational and fixpoint semantics coincide.

Definition 22. Two generative SPARQL patterns $\mathcal{P}$ and $\mathcal{P}'$ are equivalent, written $\mathcal{P} \equiv \mathcal{P}'$, if they produce the same set of solution mappings for every RDF graph $\mathcal{G}$ and every context-supply set S , i.e., $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S = \llbracket \mathcal{P}' \rrbracket_{\mathcal{G}}^S$.

SPARQL-specific equivalences. Standard SPARQL equivalences that preserve the context supplied to GenOp patterns remain valid:

- $\mathcal{P}_1 \text{ Join } \mathcal{P}_2 \equiv \mathcal{P}_2 \text{ Join } \mathcal{P}_1$;
- $(\mathcal{P}_1 \text{ Join } \mathcal{P}_2) \text{ Join } \mathcal{P}_3 \equiv \mathcal{P}_1 \text{ Join } (\mathcal{P}_2 \text{ Join } \mathcal{P}_3)$;
- $\mathcal{P}_1 \text{ Join } (\mathcal{P}_2 \text{ Union } \mathcal{P}_3) \equiv (\mathcal{P}_1 \text{ Join } \mathcal{P}_2) \text{ Union } (\mathcal{P}_1 \text{ Join } \mathcal{P}_3)$;
- $\text{Filter}_{\mathcal{F}_1 \wedge \mathcal{F}_2}(\mathcal{P}) \equiv \text{Filter}_{\mathcal{F}_1}(\text{Filter}_{\mathcal{F}_2}(\mathcal{P}))$;
- $\mathcal{P}_1 \text{ Minus } \mathcal{P}_2 \equiv \mathcal{P}_1$ if $\text{var}(\mathcal{P}_1) \cap \text{var}(\mathcal{P}_2) = \emptyset$.

The proofs follow those of SPARQL, with equality on shared variables replaced by compatibility.

GenOp-specific rewrites. A pattern $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$ can be evaluated only using context mappings that bind all variables in X ; moving g can therefore change the context set and the result.

We first show why unrestricted reordering of GenOp patterns is unsound.

Example 3. Let $g = \text{GenOp}(\pi(\{?x\}), \langle ?y \rangle, \mathcal{M})$, let $\mathcal{P}_1$ bind $?x$, and choose a GenOp-free $\mathcal{P}_2$ such that $\llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}} \neq \emptyset$ but $\llbracket \mathcal{P}_1 \text{ Join } \mathcal{P}_2 \rrbracket_{\mathcal{G}} = \emptyset$. Compare the standard distributivity forms

$$Q_L = \mathcal{P}_1 \text{ Join } (\mathcal{P}_2 \text{ Union } g) \text{ and}$$

$$Q_R = (\mathcal{P}_1 \text{ Join } \mathcal{P}_2) \text{ Union } (\mathcal{P}_1 \text{ Join } g).$$

For Q_L , deleting g yields $\mathcal{P}_1 \text{ Join } \mathcal{P}_2$, so $S_g = \emptyset$. For Q_R , deleting g yields $(\mathcal{P}_1 \text{ Join } \mathcal{P}_2) \text{ Union } \mathcal{P}_1$, whose evaluation contains the mappings of $\mathcal{P}_1$ that bind $?x$; hence S_g may be nonempty and g may produce $?y$. Thus, a standard rewrite is sound only when it preserves the context supplied to each GenOp occurrence.

We next give sufficient conditions guaranteeing that common rewrites involving GenOp patterns are sound. For a base-mode GenOp of the form $g = \text{GenOp}(\pi(\emptyset), Y, \mathcal{M})$, we use the singleton context supply $\{\emptyset\}$, independently of the supplied context set S .

Proposition 3. *Let $g = \text{GenOp}(\pi(\emptyset), Y, \mathcal{M})$ be a base-mode GenOp. Then, $\forall \mathcal{G}, \forall S : S_g = \{\emptyset\}$ and $\llbracket g \rrbracket_{\mathcal{G}}^{S_g} = \llbracket g \rrbracket_{\mathcal{G}}^{\{\emptyset\}}$.*

Corollary 1. *Let $g = \text{GenOp}(\pi(\emptyset), Y, \mathcal{M})$ and $\Omega_g = \llbracket g \rrbracket_{\mathcal{G}}^{\{\emptyset\}}$. Let $\mathcal{P}$ be a query pattern that contains no GenOp occurrences. Then, $\forall \mathcal{G}, \forall S, S' :$*

$$\llbracket \mathcal{P} \text{ Join } g \rrbracket_{\mathcal{G}}^S = \llbracket \mathcal{P} \text{ Join } g \rrbracket_{\mathcal{G}}^{S'} = \llbracket \mathcal{P} \text{ Join } \Omega_g \rrbracket_{\mathcal{G}}.$$

Proposition 4. *Let $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$ and let $\mathcal{F}$ be a filter with $\text{var}(\mathcal{F}) \subseteq X$. Assume that $\mathcal{F}$ contains no atom of the form $(v = v')$. Let $\mathcal{P}$ be a pattern with $Y \cap \text{var}(\mathcal{P}) = \emptyset$, $\text{var}(\mathcal{F}) \subseteq \text{var}(\mathcal{P})$ and $\forall \mu \in \llbracket \mathcal{P} \rrbracket_{\mathcal{G}} : X \subseteq \text{dom}_{\mu}$. Then,*

$$\llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P}) \text{ Join } g \rrbracket_{\mathcal{G}} = \llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P} \text{ Join } g) \rrbracket_{\mathcal{G}}.$$

Proposition 5 (Safe Join Reordering). *Let $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$ and let $\mathcal{P}_1, \mathcal{P}_2$ be patterns such that $X \subseteq \text{dom}_{\mu_1}$ for every $\mu_1 \in \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}}$, $\text{var}(\mathcal{P}_2) \cap X = \emptyset$, and $\llbracket \mathcal{P}_2 \rrbracket_{\mathcal{G}}$ is closed under $\cup$ for compatible mappings. Then, for all graph $\mathcal{G}$ and all context-supply set S ,*

$$\llbracket (\mathcal{P}_1 \text{ Join } \mathcal{P}_2) \text{ Join } g \rrbracket_{\mathcal{G}}^S = \llbracket (\mathcal{P}_1 \text{ Join } g) \text{ Join } \mathcal{P}_2 \rrbracket_{\mathcal{G}}^S.$$

Topological evaluation orderings preserve semantics when the generative dependency graph is acyclic.

Theorem 6. *Let $\mathcal{P}$ be a pattern whose generative dependency graph $\vec{\mathcal{D}}_{\mathcal{P}}$ is acyclic. Let γ_1 and γ_2 be two topological orders of $\text{gen}(\mathcal{P})$. For each order, let Ω_{γ_1} (resp. Ω_{γ_2}) be the set of typed solution mappings*

obtained by evaluating all GenOp occurrences according to that order, starting from the same non-generative base evaluation. Then, $\Omega_{Y_1} = \Omega_{Y_2}$.

6 Executable Fixpoint Evaluation

Recursive evaluation is declaratively given by a fixpoint construction. With generative patterns, consequences depend on finite-valued external model calls. We therefore use candidate-model validation for executable fixpoint evaluation, following the treatment of external operators in logic programs [12].

Let $\mathcal{P}$ be a monotone query pattern with immediate consequence operator $\Phi_{\mathcal{P}}$. Declaratively, the meaning of $\mathcal{P}$ is $\text{lfp}(\Phi_{\mathcal{P}})$. Executable evaluation checks SCC-local self-support on finitely many candidates; soundness with respect to the self-support target holds unconditionally, while completeness is relative to candidate coverage.

Evaluation Roadmap. Query evaluation proceeds by invoking $\text{EvalSCC}(\mathcal{G}, \mathcal{P}, C, \eta, K, B, T, R)$ (Algorithm 1) for each SCC C of $\mathcal{P}$ and each outer binding η . For a fixed call, EvalSCC first identifies the generated variables Y_C and required placeholders X_C (Sect. 6.1), then uses bounded proposal (K, B) to enumerate a finite candidate set $\text{Cand}_C(\eta) \subseteq \Omega_{\mathcal{P}}$ (Sect. 6.2). Each candidate is subsequently checked for self-support by repeated validation (R) , with optional bounded repairs (T) . The output $R_C(\eta)$ consists exactly of those candidates that satisfy self-support.

6.1 SCC-local view and self-support

Recursion in generative SPARQL arises only through generative patterns. We therefore localize recursion using strongly connected components (SCCs).

Let C be a SCC of the pattern $\mathcal{P}$, i.e., $C \subseteq \text{gen}(\mathcal{P})$. Let

$$Y_C = \bigcup_{g \in C} \text{out}(g) \quad \text{and} \quad X_C = \left(\bigcup_{g \in C} \text{input}(g) \right) \setminus Y_C,$$

where Y_C are the variables generated within C , while X_C are the external placeholders required by operators in C . An *outer binding* for C is a mapping η with $X_C \subseteq \text{dom}_{\eta}$ and $\text{dom}_{\eta} \cap Y_C = \emptyset$.

Definition 23. Let C be an SCC of $\mathcal{P}$. A mapping $\mu \in \Omega_{\mathcal{P}}$ is *self-supporting for C* (denoted $\text{SS}_C(\mu)$) if $X_C \subseteq \text{dom}_{\mu}$ and for every generative pattern $g = \text{GenOp}(\pi(X), Y, \mathcal{M}) \in C$, $\mu|_{X \cup Y} \in \llbracket g \rrbracket_{\mathcal{G}}^{\{\mu|_X\}}$.

The condition $\text{SS}_C(\mu)$ requires that each GenOp in the SCC reproduces the bindings already present in μ when evaluated on the placeholder bindings induced by μ itself. This condition replaces incremental rule firing as the operational criterion for recursive admissibility.

6.2 Executable Interface

We separate executable evaluation into *proposal*, which heuristically enumerates a finite search space, and *validation*, which enforces semantic correctness.

- (1) *Validation Oracle.* For a pattern $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$ and a *total* placeholder binding θ over X , we assume a Boolean procedure

$$\text{Validate}(g, \theta, v; R) \in \{\top, \perp\}$$

that returns $\top$ iff, under fixed decoding, parsing, and normalization settings repeated R times, the extension $(\theta \uplus v)$ belongs to $\llbracket g \rrbracket_{\mathcal{G}}^{\{\theta\}}$. Thus, Validate implements a bounded membership test for the denotation of g on fully instantiated prompts.

- (2) *Proposal Routine.* $\text{Propose}(g, \theta, K)$ returns up to K candidate bindings v for variables in $\text{out}(g) \cap Y_C$ under placeholder binding θ . Proposal may be invoked on partially instantiated contexts by filling missing placeholders with fixed sentinel tokens (or using fill-in-the-blank prompting [11]); sentinels serve only to diversify candidate enumeration and have no semantic status. Only fully instantiated bindings are validated.
- (3) *Candidate sets and executable SCC answers.* Let C be an SCC and η an outer binding. For each $v \in Y_C$, maintain a finite domain $D[v] \subseteq \mathcal{T}_{\text{gen}} \times \{\text{gen}\}$ with $|D[v]| \leq B$. These domains induce a finite candidate set

$$\text{Cand}_C(\eta) = \{\eta \uplus v \mid v : Y_C \rightarrow \mathcal{T}_{\text{gen}} \times \{\text{gen}\},$$

$$\forall v \in Y_C : v(v) \in D[v], \eta \sim v\}.$$

By construction, $\text{Cand}_C(\eta) \subseteq \Omega_{\mathcal{P}}$. The candidate set is an operational search space and is not defined in terms of the consequence operator Φ_C ; it may therefore contain mappings that are not semantically admissible. The outer binding η can be viewed as a singleton *context supply* $\{\eta\} \subseteq \Omega_{\mathcal{P}}$, consistent with the general evaluation semantics. Restriction of context supplies to placeholders (notation $S|_g$) is not used during candidate construction, since no generative operators are semantically evaluated at that stage. Such restrictions arise only during validation, where each candidate μ induces the placeholder binding $\mu|_X$ used to check membership $\mu|_{X \cup Y} \in \llbracket g \rrbracket_{\mathcal{G}}^{\{\mu|_X\}}$. Given a finite candidate set $\text{Cand}_C(\eta)$, let

$$R_C(\eta) = \{\mu|_{Y_C} \mid \mu \in \text{Cand}_C(\eta) \wedge \text{SS}_C(\mu)\}.$$

Thus, executable evaluation $R_C(\eta)$ returns exactly the Y_C -projections of candidates that satisfy self-support.

Finite candidate coverage and exactness assumption. For every SCC C of the generative-dependency graph and every outer binding η , the proposal step yields a finite candidate set $\text{Cand}_C(\eta)$. For the chosen semantics $\text{Sem} \in \{\text{lfp}, \text{stratified}, \text{well-founded}\}$, the accepted projections $R_C(\eta)$ are exactly the declarative answer projections for C under η . All generated terms, parsed tuples, candidates, and validation inputs have polynomial-size representations.

6.3 Algorithm EvalSCC

Algorithm 1 constructs finite domains $D[\cdot]$ by proposal, forms $\text{Cand}_C(\eta)$, and filters it by self-support validation. All semantic checks occur only during validation, on fully instantiated mappings.

Guarantees. For any finite candidate set $\text{Cand}_C(\eta)$, Algorithm 1 returns exactly the candidate projections that pass the local self-support validation. Thus, the algorithm is exact on the chosen candidate set, sound relative to the self-support target, and complete when the candidate set covers all self-supporting extensions of η . Under the finite candidate coverage and exactness assumption above, this target coincides with the declarative answer projections for C under η . For fixed bounds (K, B, T, R) , the algorithm terminates; see Appendix D.

Algorithm 1 EvalSCC($\mathcal{G}, \mathcal{P}, C, \eta, K, B, T, R$)

Require: RDF graph $\mathcal{G}$; pattern $\mathcal{P}$; SCC C ; outer binding η with $X_C \subseteq \text{dom}(\eta)$; bounds K (proposals), B (domain cap), T (repairs), R (validation repetitions).

Ensure: $R_C(\eta) = \{\mu|_{Y_C} \mid \mu \in \text{Cand}_C(\eta) \wedge \text{SS}_C(\mu)\}$.

- 1: **Candidate enumeration (heuristic).**
- 2: Initialize $D[v] \leftarrow \emptyset$ for all $v \in Y_C$.
- 3: **while** not saturated and $\exists v \in Y_C : |D[v]| < B$ **do**
- 4: **for all** $g = \text{GenOp}(\pi(X_g), Y_g, \mathcal{M}) \in C$ **do**
- 5: Construct a possibly partial context θ extending η ; fill any missing placeholders in X_g with fixed sentinel tokens.
- 6: $A \leftarrow \text{Propose}(g, \theta, K)$.
- 7: **for all** $(v, a) \in A$ with $v \in Y_g \cap Y_C$ **do**
- 8: **if** $|D[v]| < B$ **then**
- 9: $D[v] \leftarrow D[v] \cup \{a\}$.
- 10: **end if**
- 11: **end for**
- 12: **end for**
- 13: **end while**
- 14: **Candidate construction.**
- 15: Form $\text{Cand}_C(\eta)$ from $D[\cdot]$ as complete mappings extending η (optional symbolic pruning).
- 16: **Backtracking search with validation.**
- 17: $R_C(\eta) \leftarrow \emptyset$.
- 18: **for all** complete candidates $\mu \in \text{Cand}_C(\eta)$ **do**
- 19: *Self-support validation:*
- 20: **if** for every $g = \text{GenOp}(\pi(X_g), Y_g, \mathcal{M}) \in C$, with $v_g := \mu|_{Y_g}$, we have $\text{dom}(v_g) = Y_g$, $\mu|_{X_g \cup Y_g} = \mu|_{X_g} \uplus v_g$, and $\text{Validate}(g, \mu|_{X_g}, v_g; R) = \top$ **then**
- 21: $R_C(\eta) \leftarrow R_C(\eta) \cup \{\mu|_{Y_C}\}$.
- 22: **else**
- 23: Optionally attempt up to T repairs by resampling values for a failing variable via *Propose* and re-validating.
- 24: **end if**
- 25: **end for**
- 26: **return** $R_C(\eta)$

6.4 Evaluation via SCC-condensation

Each SCC C is summarized by a derived macro-operator GenSCC_C from X_C to Y_C , with denotation

$$\llbracket \text{GenSCC}_C \rrbracket_{\mathcal{G}}^S = \{\eta \uplus \rho \mid \eta \in S, X_C \subseteq \text{dom}(\eta), \rho \in R_C(\eta), \eta \sim \rho\}.$$

where $R_C(\eta)$ is computed by Algorithm 1. Under the finite candidate coverage and exactness assumption, replacing each SCC by its macro-operator preserves acyclic and stratified evaluation. The resulting SCC condensation is acyclic and is evaluated in topological order.

7 Computational Complexity

We analyze the complexity of query evaluation under the semantics of Sections 3 and 4. We abstract from the internal cost of model inference and treat each GenOp as an external oracle, considering only symbolic query evaluation.

We assume the deterministic bounded-generation, finite-candidate-coverage, and exactness conditions, and abstract from generative-model-call and parsing costs, treating them as $O(1)$ per invocation, while assuming that these operations are uniformly simulable by an ordinary polynomial-space machine. The computational task is evaluation membership: given a graph pattern $\mathcal{P}$, an RDF graph $\mathcal{G}$, a typed solution mapping μ , and the applicable semantics (acyclic, stratified, or well-founded), decide whether μ belongs to the semantic value of $\mathcal{P}$ over $\mathcal{G}$, i.e., whether $\mu \in \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}$.

Theorem 7. *Under deterministic bounded generation and the finite candidate coverage and exactness conditions, the evaluation-membership problem satisfies the following bounds:*

Semantics	Data complexity	Combined complexity
Acyclic	PTIME	PSPACE-complete
Stratified	PTIME	PSPACE-complete
Well-founded	PTIME	PSPACE-hard; in EXPTIME

Bounded generation induces a finite universe of typed solution mappings. For a fixed pattern, this universe is polynomial in the size of the RDF graph, which yields the PTIME data bounds in all three cases. In combined complexity, its size may be exponential. Acyclic and stratified membership can nevertheless be evaluated depth-first in polynomial space, while the standard SPARQL fragment already gives the matching PSPACE lower bound.

Under well-founded semantics, evaluation performs the alternating fixpoint construction over the finite mapping universe. This yields the stated EXPTIME combined upper bound, while the GenOp -free SPARQL fragment establishes PSPACE-hardness. Thus, under the stated assumptions, well-founded evaluation remains in PTIME in data complexity and decidable in EXPTIME in combined complexity.

8 Conclusions and Future Work

We presented a conservative, declarative extension of SPARQL with a generative pattern, GenOp . The proposed query semantics accommodates acyclic, stratified, and non-stratified generative dependencies via consequence operators and fixpoint constructions. Under deterministic bounded generation, evaluation retains PTIME data complexity; acyclic and stratified evaluation is PSPACE-complete in combined complexity, while well-founded evaluation is PSPACE-hard and in EXPTIME.

The only form of recursion supported arises from dataflow dependencies among generative patterns, where the outputs of one GenOp parameterize the prompt of another. This recursion operates at the level of prompt instantiation rather than fact derivation, and no generated value becomes part of the RDF graph under query. As a result, the language extends SPARQL with controlled generative dependencies while retaining its fixed-dataset semantics and the bounded-evaluation guarantees stated above [2] and remaining fundamentally different from Datalog-style rule languages [5, 6, 15] in terms of both expressiveness and complexity.

Future work includes relaxing bounded generation and adding Datalog-style existential predicates to GenOp . We also plan to characterize decidable guarded or acyclic fragments [4, 10, 16] and develop optimizer support for generative operators within the SPARQL algebra.

References

- [1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of databases*. Vol. 8. Addison-Wesley Reading.
- [2] Marcelo Arenas, Claudio Gutierrez, and Jorge Pérez. 2009. Foundations of RDF databases. In *Reasoning Web International Summer School*. Springer, 158–204.
- [3] Marcelo Arenas and Jorge Pérez. 2011. Querying semantic web data with SPARQL. In *Proceedings of the thirtieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 305–316.
- [4] Marco Calautti, Georg Gottlob, and Andreas Pieris. 2015. Chase termination for guarded existential rules. In *Proceedings of the 34th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. 91–103.
- [5] Andrea Cali, Georg Gottlob, Thomas Lukasiewicz, Bruno Marnette, and Andreas Pieris. 2010. Datalog+/-: A family of logical knowledge representation and query languages for new applications. In *2010 25th annual IEEE symposium on logic in computer science*. IEEE, 228–242.
- [6] Andrea Cali, Georg Gottlob, and Andreas Pieris. 2010. Query answering under non-guarded rules in Datalog+/- . In *International Conference on Web Reasoning and Rule Systems*. Springer, 1–17.
- [7] Melisachew Wudage Chekol. 2016. On the containment of SPARQL queries under entailment regimes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 30.
- [8] Edgar F Codd. 1970. A relational model of data for large shared data banks. *Commun. ACM* 13, 6 (1970), 377–387.
- [9] Hanjun Dai, Bethany Wang, Xingchen Wan, Bo Dai, Sherry Yang, Azade Nova, Pengcheng Yin, Mangpo Phothilimthana, Charles Sutton, and Dale Schuurmans. 2024. UQE: A Query Engine for Unstructured Databases. *Advances in Neural Information Processing Systems* 37 (2024), 29807–29838.
- [10] Alin Deutsch, Alan Nash, and Jeff Remmel. 2008. The chase revisited. In *Proceedings of the twenty-seventh ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 149–158.
- [11] Chris Donahue, Mina Lee, and Percy Liang. 2020. Enabling Language Models to Fill in the Blanks. arXiv:2005.05339 [cs.CL] <https://arxiv.org/abs/2005.05339>.
- [12] Thomas Eiter, Giovambattista Ianni, Roman Schindlauer, and Hans Tompits. 2006. Effective integration of declarative rules with external evaluations for semantic-web reasoning. In *European Semantic Web Conference*. Springer, 273–287.
- [13] Sebastián Ferrada, Benjamin Bustos, and Aidan Hogan. 2024. Similarity joins and clustering for SPARQL. *Semantic Web* 15, 5 (2024), 1701–1732.
- [14] Daniela Florescu, Alon Levy, and Alberto Mendelzon. 1998. Database techniques for the World-Wide Web: A survey. *ACM Sigmod Record* 27, 3 (1998), 59–74.
- [15] Georg Gottlob, Thomas Lukasiewicz, and Andreas Pieris. 2014. Datalog+/-: Questions and Answers.. In *KR*.
- [16] Georg Gottlob, Michael Morak, and Andreas Pieris. 2015. Recent Advances in Datalog. *Reasoning Web International Summer School* (2015), 193–217.
- [17] Todd J Green, Shan Shan Huang, Boon Thau Loo, Wenchao Zhou, et al. 2013. Datalog and recursive query processing. *Foundations and Trends® in Databases* 5, 2 (2013), 105–195.
- [18] Olaf Hartig. 2013. SQUIN: a traversal based query execution system for the web of linked data. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. 1081–1084.
- [19] Olaf Hartig, Christian Bizer, and Johann-Christoph Freytag. 2009. Executing SPARQL queries over the web of linked data. In *International semantic web conference*. Springer, 293–309.
- [20] Olaf Hartig and Jorge Pérez. 2016. LDQL: A query language for the web of linked data. *Journal of Web Semantics* 41 (2016), 9–29.
- [21] Inma Hernández, Carlos R Rivero, and David Ruiz. 2019. Deep Web crawling: a survey. *World Wide Web* 22, 4 (2019), 1577–1610.
- [22] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. 2021. Knowledge graphs. *ACM Computing Surveys (Csur)* 54, 4 (2021), 1–37.
- [23] Mark Kaminski, Egor V Kostylev, and Bernardo Cuenca Grau. 2017. Query nesting, assignment, and aggregation in SPARQL 1.1. *ACM Transactions on Database Systems (TODS)* 42, 3 (2017), 1–46.
- [24] Elham Khabiri, Jeffrey O Kephart, Fenno F Heath Iii, Srideepika Jayaraman, Yingjie Li, Fateh A Tipu, Dhruv Shah, Achille Fokoue-Nkoutche, and Anu Bhamidipaty. 2025. Declarative Techniques for NL Queries over Heterogeneous Data. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*. 1744–1761.
- [25] Christoph Kiefer, Abraham Bernstein, and Markus Stocker. 2007. The fundamentals of iSPARQL: A virtual triple approach for similarity-based semantic web tasks. In *International Semantic Web Conference*. Springer, 295–309.
- [26] Roman Kontchakov and Egor V Kostylev. 2016. On Expressibility of Non-Monotone Operators in SPARQL.. In *KR*. 369–379.
- [27] Frank Manola, Eric Miller, Brian McBride, et al. 2004. RDF primer. *W3C recommendation* 10, 1-107 (2004), 6.
- [28] Pascal Molli, Hala Skaf-Molli, Sebastian Ferré, Alban Gaignard, and Peggy Cellier. 2025. SparqLLM: Retrieval-Augmented SPARQL Query Processing. In *European Semantic Web Conference*. Springer, 103–107.
- [29] Ilkka Niemelä. 1999. Logic programs with stable model semantics as a constraint programming paradigm. *Annals of mathematics and Artificial Intelligence* 25, 3 (1999), 241–273.
- [30] Kanghee Park, Timothy Zhou, and Loris D’Antoni. 2025. Flexible and Efficient Grammar-Constrained Decoding. arXiv:2502.05111 [cs.CL] <https://arxiv.org/abs/2502.05111>.
- [31] Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. 2006. Semantics and Complexity of SPARQL. In *International semantic web conference*. Springer, 30–43.
- [32] Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. 2009. Semantics and complexity of SPARQL. *ACM Transactions on Database Systems (TODS)* 34, 3 (2009), 1–45.
- [33] Eric Prud’hommeaux and Andy Seaborne. 2008. SPARQL query language for RDF. W3C Recommendation, January 15, 2008. <https://www.w3.org/TR/rdf-sparql-query/> (2008).
- [34] Teodor Przymusiński. 1990. Well-founded semantics coincides with three-valued stable semantics. *Fundamenta informaticae* 13, 4 (1990), 445–463.
- [35] Mohammed Saeed, Nicola De Cao, and Paolo Papotti. 2024. Querying Large Language Models with SQL. In *EDBT*.
- [36] Michael Schmidt, Michael Meier, and Georg Lausen. 2010. Foundations of SPARQL query optimization. In *Proceedings of the 13th international conference on database theory*. 4–33.
- [37] Jie Tan, Kangfei Zhao, Rui Li, Jeffrey Xu Yu, Chengzhi Piao, Hong Cheng, Helen Meng, Deli Zhao, and Yu Rong. 2025. Can large language models be query optimizer for relational databases? *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–28.
- [38] Jeffrey D Ullman. 1988. *Principles of Computer and Knowledge-base Systems*. Computer Science Press.
- [39] Jürgen Umbrich, Aidan Hogan, Axel Polleres, and Stefan Decker. 2015. Link traversal querying for a diverse web of data. *Semantic Web* 6, 6 (2015), 585–624.
- [40] Allen Van Gelder. 1989. The alternating fixpoint of logic programs with negation. In *Proceedings of the eighth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*. 1–10.
- [41] Wenli Yang, Lilian Some, Michael Bain, and Byeong Kang. 2025. A comprehensive survey on integrating large language models with knowledge-based methods. *Knowledge-Based Systems* (2025), 113503.
- [42] Fuheng Zhao, Divyakant Agrawal, and Amr El Abbadi. 2024. Hybrid Querying Over Relational Databases and Large Language Models. (2024). arXiv:2408.00884 [cs.DB] <https://arxiv.org/abs/2408.00884>
- [43] Weiguo Zheng, Lei Zou, Wei Peng, Xifeng Yan, Shaoxu Song, and Dongyan Zhao. 2016. Semantic SPARQL similarity search over RDF knowledge graphs. *Proceedings of the VLDB Endowment* 9, 11 (2016), 840–851.

Extended: Running Example

A Sample run of Example 1

Below, we illustrate the executable treatment of mutually dependent GenOp occurrences. The run constructs a candidate mapping and checks whether it is locally self-supporting. Correctness with respect to the declarative component answers is relative to the finite candidate coverage and exactness assumption from Sect. 6.

Example 4. Let $\mathcal{G} = \{:\text{Paris} \text{ a } :\text{City}\}$ and consider the query

$$Q = (\text{BGP Join } g_1) \text{ Join } g_2,$$

where

- $\text{BGP} = \{?x \text{ a } :\text{City}\},$
- $g_1 = \text{GenOp}(\pi_1(\{?x, ?z\}), \langle ?y \rangle, \text{GPT-4o})$ with $\pi_1(\{?x, ?z\}) =$ “Suggest a topic related to ?x and ?z”, and
- $g_2 = \text{GenOp}(\pi_2(\{?y\}), \langle ?z \rangle, \mathcal{M}_2),$
where $\mathcal{M}_2 = \text{Gemini 1.5 Pro}$ and

$$\pi_2(\{?y\}) = \text{“Suggest a city related to topic ?y”}.$$

Evaluating the basic graph pattern gives

$$S^{(0)} = \llbracket \{?x \text{ a } :\text{City}\} \rrbracket_{\mathcal{G}} = \{\mu_0\}, \mu_0 = \{?x \mapsto (: \text{Paris}, \text{rdf})\}.$$

Neither g_1 nor g_2 is applicable to μ_0 in a single pass: g_1 requires a binding for $?z$, while g_2 requires a binding for $?y$. Thus, the dependency graph of Q contains a cycle between g_1 and g_2 . Executable evaluation handles this cycle by considering finite candidate extensions of μ_0 and validating them against the two local generative conditions.

We now exhibit one candidate mapping and the validation witnesses for it.

Hypothesized binding and instantiation of g_2 .

Assume that a candidate mapping μ^* binds $?y$ to the generated term (t_A, gen) with

$$\text{Lex}(t_A) = \text{"art"}.$$

Instantiating π_2 with the input projection of μ^* gives

$$\pi_2^{\lfloor \mu_{\{?y\}}^* \rfloor} = \text{"Suggest a city related to topic art"}.$$

Assume

$$\text{Call}_{\text{Gemini 1.5 Pro}}(\pi_2^{\lfloor \mu_{\{?y\}}^* \rfloor}) = \{\text{"Florence"}\},$$

and that

$$\begin{aligned} \text{Parse}_{\text{Gemini 1.5 Pro}}(\text{"Florence"}) &= \{(t_F)\}, \\ \text{Lex}(t_F) &= \text{"Florence"}, \end{aligned}$$

This validates the generated binding

$$\{?z \mapsto (t_F, \text{gen})\}$$

for g_2 under the input binding $?y \mapsto (t_A, \text{gen})$.

Instantiation of g_1 .

Given the bindings for $?x$ and $?z$, instantiating π_1 gives

$$\pi_1^{\lfloor \mu_{\{?x, ?z\}}^* \rfloor} = \text{"Suggest a topic related to Paris and Florence"}.$$

Assume

$$\text{Call}_{\text{GPT-4o}}(\pi_1^{\lfloor \mu_{\{?x, ?z\}}^* \rfloor}) = \{\text{"art"}\},$$

and

$$\text{Parse}_{\text{GPT-4o}}(\text{"art"}) = \{(t_A)\} \quad \text{and} \quad \text{Lex}(t_A) = \text{"art"}.$$

This validates the generated binding

$$\{?y \mapsto (t_A, \text{gen})\}$$

for g_1 under the input bindings $?x \mapsto (: \text{Paris}, \text{rdf})$ and $?z \mapsto (t_F, \text{gen})$.

Let

$$\begin{aligned} \mu^* &= \{?x \mapsto (: \text{Paris}, \text{rdf}), ?y \mapsto (t_A, \text{gen}), \\ &\quad ?z \mapsto (t_F, \text{gen})\}. \end{aligned}$$

Self-support validation. The mapping μ^* extends μ_0 and binds the input and output variables of both generative occurrences. For g_1 , the projection $\mu_{\{?x, ?z\}}^*$ instantiates the prompt, and the projection $\mu_{\{?y\}}^*$ is validated as an admissible output. For g_2 , the projection $\mu_{\{?y\}}^*$ instantiates the prompt, and the projection $\mu_{\{?z\}}^*$ is validated as an admissible output. Hence μ^* passes the local self-support test for the component $\{g_1, g_2\}$.

If μ^* belongs to the finite candidate set considered by the executable evaluator, then the evaluator accepts the projection $\mu_{\{?y, ?z\}}^*$. Under the finite candidate coverage and exactness assumption

from Sect. 6, this accepted projection coincides with a declarative component-answer projection. The resulting full query answer is

$$\{?x \mapsto (: \text{Paris}, \text{rdf}), ?y \mapsto (t_A, \text{gen}), ?z \mapsto (t_F, \text{gen})\}.$$

The generated bindings $?y$ and $?z$ are typed with gen and they do not add RDF triples to $\mathcal{G}$.

A.1 Executable Fixpoint Evaluation of Example 1

We now demonstrate executable fixpoint evaluation by bounded candidate enumeration followed by local self-support validation.

We illustrate the executable procedure from Sect. 6 on Example 1, making explicit (i) how Propose induces a finite candidate set and (ii) how validation checks candidate mappings on fully instantiated prompts. We separate *candidate discovery* from *candidate validation*: Propose determines which candidates are explored, while validation determines which explored candidates are accepted. Correctness with respect to the declarative component answers is relative to the finite candidate coverage and exactness assumption from Sect. 6.

Recall Example 1,

```
SELECT ?x ?y ?z WHERE {
  ?x a :City .
  GenOp("Suggest a topic related to ?x and ?z" AS ?y,
        GPT-4o).
  GenOp("Suggest a city related to topic ?y"
        AS ?z, Gemini 1.5 Pro).
}
Let  $\mathcal{G} = \{:\text{Paris} \ a \ : \text{City}\}$ . Let
```

$$\begin{aligned} g_1 &= \text{GenOp}(\pi_1(\{?x, ?z\}), \langle ?y \rangle, \mathcal{M}_1), \\ g_2 &= \text{GenOp}(\pi_2(\{?y\}), \langle ?z \rangle, \mathcal{M}_2), \end{aligned}$$

where

- π_1 is the template "Suggest a topic related to $?x$ and $?z$ "; and
- π_2 is "Suggest a city related to topic $?y$ ".

We assume canonical lexicalization for RDF terms. For example, $\text{Canon}(: \text{Paris}) = \text{"Paris"}$. Generated terms $t \in \mathcal{T}_{\text{gen}}$ carry lexicalizations $\text{Lex}(t) \in \Sigma^*$.

Assume the following operational bounds and decoding-format conventions.

- (1) Proposal bound K (i.e., maximum proposals per Propose call) and domain cap B (i.e., maximum per-variable domain size). Here, K bounds the number of distinct values returned by a single proposal call, while B bounds the total number of values accumulated for each generated variable across proposal calls. Together, K and B make the explored candidate domains finite. Values not proposed because of these bounds are not explored operationally; they are part of the declarative semantics only when covered by the finite candidate coverage assumption.
- (2) Validation repetitions R and optional repair attempts T . Validation uses a binary membership-style query to the model. The parameter R bounds repeated validation calls used by an implementation to stabilize the acceptance decision. If validation fails, T bounds the number of local repair attempts in which an alternative generated value is substituted and revalidated.

These parameters affect the executable procedure and runtime; the declarative semantics is fixed independently of them.

- (3) The model response must be parseable as JSON. Grammar-constrained decoding can enforce this format [30]. For g_1 (output variable $?y$), we use the schema

$$\{ "y": ["<string>", \dots] \},$$

and for g_2 (output variable $?z$), we use

$$\{ "z": ["<string>", \dots] \}.$$

- (4) Decoding parameters, such as temperature, top- p , and maximum number of tokens, are fixed per model and treated as part of the implementation.

Parsing maps each output string s to a fresh generated term $t_s \in \mathcal{T}_{\text{gen}}$ with $\text{Lex}(t_s) = s$, or reuses an existing t_s if already created.

A.1.1 Propose and Self-validation Procedures. We instantiate the abstract semantics with two concrete operational procedures.

(A) *Propose as bounded enumeration.* For $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$, context mapping S , and bound K :

- (1) *Prompt instantiation.* Build a concrete prompt string $\pi^{[S]}$ by replacing placeholders in $X \cap \text{dom}(S)$ using lexicalization. If some placeholder in X is unbound in S , render it using a fixed sentinel token <unknown> : ?v .
- (2) *Model call.* Query $\mathcal{M}$ under fixed decoding parameters with an instruction requiring JSON output and at most K items.
- (3) *Parse and deduplicate.* Parse the JSON, extract up to K distinct strings, and map them to generated terms deterministically.
- (4) *Return.* Return the resulting set of at most K generated-term tuples. In this example, all tuples are singletons because $|Y| = 1$.

This procedure never claims completeness; it merely returns a bounded set of candidates. Finiteness is enforced by the explicit bound K and by the domain cap B applied when inserting into $D[\cdot]$.

(B) *Self-support validation as membership checking.* Given a complete candidate mapping μ and $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$, validation checks whether the tuple $\mu|_Y$ is an admissible output for g under the fully instantiated prompt $\pi^{[\mu|_X]}$. We use a verification query, that is, a Boolean YES/NO query, rather than relying on exact re-generation:

- (1) Instantiate the base instruction prompt $\pi^{[\mu|_X]}$.
- (2) Form a verification prompt that includes the instruction and the candidate output tuple $\mu|_Y$, and asks for a strict YES/NO answer.
- (3) Query $\mathcal{M}$ under the fixed validation policy. Repeat R times if specified.
- (4) Accept if the aggregation policy for the R validation calls accepts the candidate.

This gives an explicit implementation of the abstract membership test

$$\mu|_X \sqcup \mu|_Y \in \llbracket g \rrbracket_{\mathcal{G}}^{\{\mu|_X\}}.$$

A.1.2 Memoization. Maintain two caches: (i) a proposal cache keyed by $(g, \pi^{[S]}, K, \text{decoding})$; and (ii) a validation cache keyed by $(g, \pi^{[\mu|_X]}, \mu|_Y, R)$. Caches stabilize evaluation and reduce repeated model calls.

- (1) *Step 1: Base RDF evaluation.* Evaluating the basic graph pattern

$$?x \text{ a } : \text{City}$$

over $\mathcal{G}$ yields the unique typed solution mapping

$$\mu_0 = \{ ?x \mapsto (: \text{Paris}, \text{rdf}) \}.$$

This is the outer binding $\eta := \mu_0$.

- (2) *Step 2: SCC identification and summarization.* Since $?y$ feeds g_2 and $?z$ feeds g_1 , the dependency graph contains one SCC:

$$C = \{g_1, g_2\}.$$

The externally supplied variables are

$$X_C = \{ ?x \},$$

and the variables generated inside the component are

$$Y_C = \{ ?y, ?z \}.$$

We summarize C by the macro-operator GenSCC_C . For an outer binding η , this operator returns full mappings of the form $\eta \sqcup \rho$, where ρ ranges over the accepted Y_C -projections produced by the executable evaluation of C under η . Under the finite candidate coverage and exactness assumption from Sect. 6, these accepted projections coincide with the declarative component-answer projections.

- (3) *Step 3: Candidate enumeration, operational only.* We invoke

$$\text{EvalSCC}(\mathcal{G}, \mathcal{P}, C, \eta, K, B, T, R).$$

Initialize finite domains:

$$D[?y] = \emptyset \quad \text{and} \quad D[?z] = \emptyset.$$

Round 1: propose values for ?y. We call $\text{Propose}(g_1, S_1, K)$ with

$$S_1 = \{ ?x \mapsto (: \text{Paris}, \text{rdf}) \}.$$

Since $?z \notin \text{dom}(S_1)$, the sentinel policy instantiates

$$\pi_1^{[S_1]} = \text{"Suggest a topic related to Paris and"} \\ \text{"unknown : ?z"}.$$

We query $\mathcal{M}_1$ with an instruction such as:

Return JSON $\{ "y": [\dots] \}$ with at most K short topic strings.

Suppose the model returns:

$$\{ "y": ["Art", "Cuisine"] \}.$$

Parsing yields generated terms t_{Art} and t_{Cuisine} with

$$\text{Lex}(t_{\text{Art}}) = \text{"Art"}, \quad \text{Lex}(t_{\text{Cuisine}}) = \text{"Cuisine"}.$$

Update, respecting the cap B :

$$D[?y] = \{ t_{\text{Art}}, t_{\text{Cuisine}} \}.$$

Round 2: propose values for ?z conditional on ?y. Choose a currently discovered value, for example $?y = t_{\text{Art}}$, and call $\text{Propose}(g_2, S_2, K)$ with

$$S_2 = \{ ?x \mapsto (: \text{Paris}, \text{rdf}), ?y \mapsto (t_{\text{Art}}, \text{gen}) \}.$$

Then,

$$\pi_2^{[S_2]} = \text{"Suggest a city related to topic Art"}.$$

Query $\mathcal{M}_2$ requiring JSON $\{ "z": [\dots] \}$ with at most K city strings. Suppose it returns:

$$\{ "z": ["Florence", "Rome"] \}.$$

Parsing yields t_{Florence} and t_{Rome} , and we update:

$$D[?z] = \{ t_{\text{Florence}}, t_{\text{Rome}} \}.$$

Further rounds. One can repeat conditional proposals, for example for $?y = t_{\text{Cuisine}}$, and alternate between g_1 and g_2 until no new values are added or the cap B is reached. In all cases, $|D[v]| \leq B$ is enforced by construction.

Finite candidate set. The induced candidate set is the finite product:

$$\text{Cand}_C(\eta) = \{\eta \uplus v \mid v(?y) \in D[?y], v(?z) \in D[?z]\}.$$

Suppose, in this run, $|D[?y]| = 2$ and $|D[?z]| = 2$. Then, $|\text{Cand}_C(\eta)| = 4$.

- (4) *Step 4: Candidate validation.* We enumerate candidates $\mu \in \text{Cand}_C(\eta)$ and accept those that pass local self-support validation, that is, those for which each $g \in C$ validates the output assigned by μ under the prompt instantiated from the input variables of g .

Candidate sample. Consider

$$\mu = \{?x \mapsto (: \text{Paris}, \text{rdf}), ?y \mapsto (t_{\text{Art}}, \text{gen}), \\ ?z \mapsto (t_{\text{Florence}}, \text{gen})\}.$$

Validate g_1 on μ (topic validation). We have $X = \{?x, ?z\}$ and $Y = \langle ?y \rangle$. Instantiate the base instruction prompt using $\mu|_X$:

$$\pi_1^{[\mu|_X]} = \text{"Suggest a topic related to Paris and Florence"}.$$

We then issue a verification query to M_1 such as:

Instruction: "Suggest a topic related to Paris and Florence."
Candidate topic: "Art".
Is the candidate a valid output for the instruction? Answer
YES or NO only.

Repeat according to the validation bound R and accept if the validation aggregation policy accepts. If accepted, this operationally establishes that t_{Art} is admissible for g_1 under the input projection $\mu|_X$.

Validate g_2 on μ (city validation). We have $X = \{?y\}$ and $Y = \langle ?z \rangle$. Instantiate:

$$\pi_2^{[\mu|_X]} = \text{"Suggest a city related to topic Art"}.$$

Verification query to M_2 :

Instruction: "Suggest a city related to topic Art."
Candidate city: "Florence".
Is the candidate a valid output for the instruction? Answer
YES or NO only.

Again, repeat according to the validation bound R and accept if the validation aggregation policy accepts.

Acceptance. The candidate μ is accepted iff both validations pass. In that case, μ satisfies the local self-support condition for the component C . If either validation fails, μ is rejected.

Optional repair. If μ fails due to, say, g_2 , we may attempt up to T repairs: keep $\mu|_X$ fixed for g_2 , here $?y = \text{Art}$, substitute an alternative $?z$ already in the fixed domain $D[?z]$, for example Rome, and rerun the same validation checks. Only mappings that pass the same local self-support validation are accepted.

- (5) *Step 5: Final result.* For each accepted candidate μ , the SCC yields $\mu|_{Y_C}$, and the full query answer is obtained by joining with the outer mapping μ_0 , which is already included in μ . Thus, answers have the form

$$\{?x \mapsto (: \text{Paris}, \text{rdf}), ?y \mapsto (t, \text{gen}), ?z \mapsto (u, \text{gen})\}.$$

Remark 2. The mechanisms Propose, sentinel placeholders, decoding choices, JSON schemas, caching, and repair only control candidate discovery and execution cost. Acceptance of a candidate is determined by local self-support validation on fully instantiated prompts. Correctness with respect to the declarative component answers is obtained under the finite candidate coverage and exactness assumption from Sect. 6.

APPENDIX: Extended Proofs and Auxiliary Results

B Fixpoint-Based Semantics

We assume a *bounded-generation* regime: for every GenOp occurrence $g = \text{GenOp}(\pi(X), Y, M)$, every instantiated prompt string produces a finite set of textual answers via Call_M , and every textual answer yields a finite set of parsed tuples via Parse_M . The total set of typed solution mappings that may be produced during evaluation is finite. Formally, the induced universe $\Omega_{\mathcal{P}}$ (as defined in the main text) is finite.

Set-theoretic lattice facts. For any set U , the powerset $(2^U, \subseteq)$ is a complete lattice with least element $\emptyset$, greatest element U , and joins given by unions. If U is finite, every ascending chain in $(2^U, \subseteq)$ stabilizes after at most $|U|$ strict increases.

Compatibility. We use the compatibility relation $\sim$ and union $\uplus$ of typed solution mappings as defined in the main text. If $\mu_1 \sim \mu_2$, then $\mu_1 \uplus \mu_2$ is well-defined; it preserves every non-overlapping binding and maps every overlapping binding to the common canonical representative, as in Definition 9.

Context restriction. For a context supply $S' \subseteq \Omega_{\mathcal{P}}$ and a GenOp occurrence g , we recall $S'|_g = \{\mu \in S' \mid \text{input}(g) \subseteq \text{dom}_\mu\}$ and the actual context supply S_g from the main text. For any context supplies $S' \subseteq S''$, we have $S'|_g \subseteq S''|_g$.

Immediate consequence operator. We use $\Phi_{\mathcal{P}} : 2^{\Omega_{\mathcal{P}}} \rightarrow 2^{\Omega_{\mathcal{P}}}$ given by $\Phi_{\mathcal{P}}(S) = \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S$ (main text). Whenever we write $\Phi_{\mathcal{P}}^n$, we mean n -fold composition, with $\Phi_{\mathcal{P}}^0$ being the identity.

B.1 Acyclic patterns: single-pass evaluation coincides with fixpoint semantics

We prove the least-fixpoint correspondence for acyclic patterns that are monotone with respect to the context supply.

Operational single-pass evaluation for acyclic patterns. Let $\vec{\mathcal{D}}_{\mathcal{P}} = (\text{gen}(\mathcal{P}), \rightarrow_{\mathcal{P}})$ be the dependency graph. Let $\vec{\mathcal{D}}$ be acyclic, and let $\gamma = (g_1, \dots, g_m)$ be a topological order of the nodes $\text{gen}(\mathcal{P})$.

For $k \in \{0, \dots, m\}$, let $\mathcal{P}_Y^{[k]}$ denote the pattern obtained from $\mathcal{P}$ by disabling every GenOp occurrence not among $\{g_1, \dots, g_k\}$, i.e., replacing it by a pattern whose evaluation yields $\emptyset$, while leaving the remaining algebraic structure unchanged. Thus, $\mathcal{P}_Y^{[0]}$ has all GenOps disabled and $\mathcal{P}_Y^{[m]} = \mathcal{P}$.

We define the staged sets by

$$S_Y^{(0)} = \llbracket \mathcal{P}_Y^{[0]} \rrbracket_{\mathcal{G}}^{\emptyset} \quad \text{and} \quad S_Y^{(k)} = \llbracket \mathcal{P}_Y^{[k]} \rrbracket_{\mathcal{G}}^{S_Y^{(k-1)}} \quad (1 \leq k \leq m).$$

At stage k , we evaluate GenOp occurrences up to g_k using the context supply accumulated at stage $(k - 1)$.

Lemma 2. (Well-defined staged evaluation) For each $k \in \{0, \dots, m\}$, the set $S_Y^{(k)}$ is well-defined and contained in $\Omega_{\mathcal{P}}$.

PROOF. By bounded generation, each GenOp call yields finitely many outputs, and every inductive evaluation clause returns a finite set of typed solution mappings. Since evaluation is defined for any supply $S \subseteq \Omega_{\mathcal{P}}$ (via S_g for each GenOp occurrence g), each $S_Y^{(k)}$ is defined. Containment in $\Omega_{\mathcal{P}}$ follows by construction. $\square$

Lemma 3. (No backward dependencies) Let $\gamma = (g_1, \dots, g_m)$ be topological. If $i < j$, then $\text{out}(g_j) \cap \text{input}(g_i) = \emptyset$.

PROOF. If $\text{out}(g_j) \cap \text{input}(g_i) \neq \emptyset$, then by definition of $\rightarrow_{\mathcal{P}}$ we have $g_j \rightarrow_{\mathcal{P}} g_i$, contradicting that γ is a topological order. $\square$

Theorem 8 (Acyclic single-pass evaluation). Assume $\vec{\mathcal{D}}_{\mathcal{P}}$ is acyclic and $\mathcal{P}$ is monotone with respect to the context supply. Let γ be a topological order, written $\gamma = (g_1, \dots, g_m)$. Then,

- (1) $S_Y^{(m)}$ is a fixpoint of $\Phi_{\mathcal{P}}$;
- (2) $S_Y^{(m)} = \text{lfp}(\Phi_{\mathcal{P}})$; and
- (3) the final staged result is independent of the chosen topological order.

PROOF. We use the following inclusion. For $0 \leq j \leq k \leq m$ and supplies $S \subseteq S'$,

$$\llbracket \mathcal{P}_Y^{[j]} \rrbracket_{\mathcal{G}}^S \subseteq \llbracket \mathcal{P}_Y^{[k]} \rrbracket_{\mathcal{G}}^{S'}. \quad (\text{A})$$

We verify (A) by structural induction on $\mathcal{P}$. A BGP is independent of the supply. For an enabled GenOp occurrence g , $S \subseteq S'$ gives $S_g \subseteq S'_g$, and Definition 10 gives inclusion of the corresponding unions. If an occurrence is disabled in $\mathcal{P}_Y^{[j]}$ and enabled in $\mathcal{P}_Y^{[k]}$, its denotation changes from $\emptyset$ to a set of mappings. Projection, union, join, and filters in positive positions preserve inclusion. By monotonicity with respect to the context supply, generated outputs do not occur in a negative position; hence no negative constructor reverses an inclusion caused by enabling a GenOp. The induction establishes (A).

Let $S_Y^{(-1)} := \emptyset$. From (A), induction on k gives

$$S_Y^{(k-1)} \subseteq S_Y^{(k)} \quad (0 \leq k \leq m). \quad (\text{B})$$

For $k = 0$, the claim is $\emptyset \subseteq S_Y^{(0)}$. If it holds for $k - 1$, then

$$S_Y^{(k-1)} = \llbracket \mathcal{P}_Y^{[k-1]} \rrbracket_{\mathcal{G}}^{S_Y^{(k-2)}} \subseteq \llbracket \mathcal{P}_Y^{[k]} \rrbracket_{\mathcal{G}}^{S_Y^{(k-1)}} = S_Y^{(k)}$$

by (A).

We next prove

$$\Phi_{\mathcal{P}}(S_Y^{(m)}) \subseteq S_Y^{(m)}. \quad (\text{C})$$

Let g_i be a GenOp occurrence. By Lemma 3, no output of g_j with $j > i$ belongs to $\text{input}(g_i)$. Hence, every prompt instantiated for g_i from $S_Y^{(m)}$ has the same restriction to $\text{input}(g_i)$ as a context available once $g_1, \dots, g_{i-1}$ have been enabled. Definition 10 depends on a context only through that restriction when constructing the prompt. Therefore, evaluating g_i from $S_Y^{(m)}$ yields no generated tuple beyond those admitted at its topological stage. Applying the

positive algebraic constructors preserves the resulting inclusion, which proves (C).

Since $\mathcal{P}_Y^{[m]} = \mathcal{P}$,

$$S_Y^{(m)} = \Phi_{\mathcal{P}}(S_Y^{(m-1)}).$$

By (B) and monotonicity of $\Phi_{\mathcal{P}}$,

$$S_Y^{(m)} = \Phi_{\mathcal{P}}(S_Y^{(m-1)}) \subseteq \Phi_{\mathcal{P}}(S_Y^{(m)}).$$

Combining the inclusion with (C) gives $\Phi_{\mathcal{P}}(S_Y^{(m)}) = S_Y^{(m)}$.

Let $L := \text{lfp}(\Phi_{\mathcal{P}})$. Since $S_Y^{(m)}$ is a fixpoint, leastness gives $L \subseteq S_Y^{(m)}$. For the reverse inclusion, induction on k yields $S_Y^{(k)} \subseteq L$. For $k = 0$, (A) with $S = \emptyset$, $S' = L$, $j = 0$, and $k = m$ gives

$$S_Y^{(0)} \subseteq \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^L = \Phi_{\mathcal{P}}(L) = L.$$

If $S_Y^{(k-1)} \subseteq L$, then (A) gives

$$S_Y^{(k)} = \llbracket \mathcal{P}_Y^{[k]} \rrbracket_{\mathcal{G}}^{S_Y^{(k-1)}} \subseteq \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^L = L.$$

Thus, $S_Y^{(m)} \subseteq L$, and therefore $S_Y^{(m)} = L$.

Every topological order yields the same least fixpoint L ; hence the final staged result is order-independent. $\square$

B.2 Context equivalence invariance

Proposition 6 (Context invariance). Let $S, S' \subseteq \Omega_{\mathcal{P}}$ be context supplies such that for every GenOp occurrence g in $\mathcal{P}$,

$$S_g = S'_g.$$

Then, $\Phi_{\mathcal{P}}(S) = \Phi_{\mathcal{P}}(S')$.

PROOF. We prove by structural induction on $\mathcal{P}$ that $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S = \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^{S'}$.

Base case 1: $\mathcal{P}$ is a BGP. Then, $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S$ is independent of S , hence equality holds.

Base case 2: $\mathcal{P}$ is a GenOp pattern $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$. By the context definition, the occurrence is evaluated using $\llbracket g \rrbracket_{\mathcal{G}}^{S_g}$ under S and $\llbracket g \rrbracket_{\mathcal{G}}^{S'_g}$ under S' . By assumption, the context sets coincide. Thus, we use the same context mappings μ_c in both evaluations. Since $\llbracket g \rrbracket_{\mathcal{G}}$ is defined as a union over all μ_c in the context supply, the two evaluations coincide. *Inductive cases.* Assume the claim holds for immediate subpatterns.

- (1) If $\mathcal{P} = \text{Proj}_L(\mathcal{P}')$, then $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S$ is obtained by restricting each mapping in $\llbracket \mathcal{P}' \rrbracket_{\mathcal{G}}^S$ to L , which preserves equality.
- (2) If $\mathcal{P} = \mathcal{P}_1 \text{ Union } \mathcal{P}_2$, then evaluation is set union of the evaluations of $\mathcal{P}_1$ and $\mathcal{P}_2$, hence equality follows.
- (3) If $\mathcal{P} = \mathcal{P}_1 \text{ Join } \mathcal{P}_2$, then evaluation is the set of unions $\mu_1 \uplus \mu_2$ for compatible μ_i from each side. By the induction hypothesis, the sets of μ_i coincide under S and S' , hence the join result coincides.
- (4) If $\mathcal{P} = \text{Filter}_F(\mathcal{P}')$, evaluation is subset selection of $\llbracket \mathcal{P}' \rrbracket_{\mathcal{G}}^S$ by a predicate on μ alone (F^μ depends only on μ). Since the underlying set is equal by the induction hypothesis, the filtered subset is equal.
- (5) The cases Opt , Diff , and Minus are analogous: each is defined purely in terms of membership and compatibility relations among mappings in the evaluated subpatterns;

equality of the evaluated sets therefore implies equality of the resulting sets.

Thus, $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S = \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^{S'}$ for all $\mathcal{P}$, and therefore $\Phi_{\mathcal{P}}(S) = \Phi_{\mathcal{P}}(S')$. $\square$

B.3 Monotonicity of the consequence operator

We prove Lemma 1 by structural induction.

Lemma 4 (Monotonicity of $\Phi_{\mathcal{P}}$). *If $\mathcal{P}$ is monotone with respect to the context supply under the condition in the main text, then $\Phi_{\mathcal{P}}$ is monotone on $(2^{\Omega_{\mathcal{P}}}, \subseteq)$:*

$$S \subseteq S' \implies \Phi_{\mathcal{P}}(S) \subseteq \Phi_{\mathcal{P}}(S').$$

PROOF. We prove the following statement: for all patterns $\mathcal{P}$ satisfying the monotonicity condition,

$$S \subseteq S' \implies \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S \subseteq \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^{S'}.$$

The proof is by structural induction on $\mathcal{P}$.

Case 1: Let $\mathcal{P}$ be a BGP. Evaluation is independent of the context supply. Hence, equality holds.

Case 2: Let $\mathcal{P} = g = \text{GenOp}(\pi(X), Y, \mathcal{M})$. We have $S_g \subseteq S'_g$ because $S \subseteq S'$. By Definition 10,

$$\llbracket g \rrbracket_{\mathcal{G}}^{S_g} \subseteq \llbracket g \rrbracket_{\mathcal{G}}^{S'_g}.$$

Case 3: Let $\mathcal{P} = \text{Proj}_L(\mathcal{P}')$. By the induction hypothesis, $\llbracket \mathcal{P}' \rrbracket_{\mathcal{G}}^S \subseteq \llbracket \mathcal{P}' \rrbracket_{\mathcal{G}}^{S'}$. Restriction to L is monotone on sets: if $A \subseteq B$ then $\{\mu|_L : \mu \in A\} \subseteq \{\mu|_L : \mu \in B\}$, hence inclusion holds.

Case 4: Let $\mathcal{P} = \mathcal{P}_1 \text{ Union } \mathcal{P}_2$. By the induction hypothesis, $\llbracket \mathcal{P}_i \rrbracket_{\mathcal{G}}^S \subseteq \llbracket \mathcal{P}_i \rrbracket_{\mathcal{G}}^{S'}$. Taking unions preserves the required inclusion.

Case 5: Let $\mathcal{P} = \mathcal{P}_1 \text{ Join } \mathcal{P}_2$. Let $\mu \in \llbracket \mathcal{P}_1 \text{ Join } \mathcal{P}_2 \rrbracket_{\mathcal{G}}^S$. Then, there exist $\mu_1 \in \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}}^S$ and $\mu_2 \in \llbracket \mathcal{P}_2 \rrbracket_{\mathcal{G}}^S$ with $\mu_1 \sim \mu_2$ and $\mu = \mu_1 \uplus \mu_2$. By the induction hypothesis, $\mu_i \in \llbracket \mathcal{P}_i \rrbracket_{\mathcal{G}}^{S'}$; the same pair therefore witnesses $\mu \in \llbracket \mathcal{P}_1 \text{ Join } \mathcal{P}_2 \rrbracket_{\mathcal{G}}^{S'}$.

Case 6: Let $\mathcal{P} = \text{Filter}_F(\mathcal{P}')$. By the induction hypothesis, $\llbracket \mathcal{P}' \rrbracket_{\mathcal{G}}^S \subseteq \llbracket \mathcal{P}' \rrbracket_{\mathcal{G}}^{S'}$. Now

$$\llbracket \text{Filter}_F(\mathcal{P}') \rrbracket_{\mathcal{G}}^S = \{\mu \in \llbracket \mathcal{P}' \rrbracket_{\mathcal{G}}^S : F^\mu = \top\}.$$

Since the predicate $F^\mu = \top$ depends only on μ , subset inclusion of the underlying sets yields inclusion of the filtered sets.

Case 7: Consider the negative constructs Minus, Diff, and Opt. The constructs are generally non-monotone in SPARQL. We consider patterns $\mathcal{P}$ satisfying the main-text monotonicity condition: no variable introduced by a GenOp output occurs in a negative position.

Under the monotonicity condition, enlarging the context supply from S to S' may introduce additional generated bindings, but the additional bindings cannot change the evaluation of any negative construct, since all negative tests range only over variables whose bindings are fixed independently of GenOp outputs. Hence, the truth of membership and compatibility checks performed by negative constructs is invariant under such enlargements.

The denotation of each negative construct is therefore unchanged when passing from S to S' , and the desired inclusion holds by the induction hypothesis applied to the positive subpatterns. The induction is complete. $\square$

B.4 Least fixpoint: existence, characterization, and minimality

Theorem 9 (Existence and characterization of the least fixpoint). *If $\Phi_{\mathcal{P}}$ is monotone on $(2^{\Omega_{\mathcal{P}}}, \subseteq)$, then:*

- (1) $\Phi_{\mathcal{P}}$ has a least fixpoint $\text{lfp}(\Phi_{\mathcal{P}})$;
- (2) $\text{lfp}(\Phi_{\mathcal{P}}) = \bigcup_{n \geq 0} \Phi_{\mathcal{P}}^n(\emptyset)$;
- (3) if $\Omega_{\mathcal{P}}$ is finite, the Kleene chain stabilizes after at most $|\Omega_{\mathcal{P}}|$ strict increases.

If Φ is ω -continuous, Item 2 also holds without the finiteness assumption.

PROOF. (1) *Existence.* The lattice $(2^{\Omega_{\mathcal{P}}}, \subseteq)$ is complete. Monotonicity of Φ and the Knaster–Tarski theorem yield the least fixpoint

$$\text{lfp}(\Phi) = \bigcap \{S \subseteq \Omega_{\mathcal{P}} \mid \Phi(S) \subseteq S\}.$$

(2) *Kleene characterization under finiteness.* Let the Kleene sequence be given by $S_0 = \emptyset$ and $S_{n+1} = \Phi(S_n)$. By monotonicity, $S_0 \subseteq S_1 \subseteq S_2 \subseteq \dots$.

Assume $\Omega_{\mathcal{P}}$ is finite. Then, $(S_n)_n$ can strictly increase at most $|\Omega_{\mathcal{P}}|$ times, hence there exists $N \leq |\Omega_{\mathcal{P}}|$ such that $S_N = S_{N+1} = \Phi(S_N)$. Thus, S_N is a fixpoint.

To show minimality, let T be any fixpoint of Φ . By induction on n , $S_n \subseteq T$ for all n : the base case is $S_0 = \emptyset \subseteq T$, and if $S_n \subseteq T$ then $S_{n+1} = \Phi(S_n) \subseteq \Phi(T) = T$ by monotonicity and $\Phi(T) = T$. Hence, $S_N \subseteq T$ for every fixpoint T ; therefore, S_N is the least fixpoint: $S_N = \text{lfp}(\Phi)$.

(3) Finally, since the chain stabilizes at N , we have $\bigcup_{n \geq 0} S_n = S_N$, and therefore

$$\text{lfp}(\Phi) = S_N = \bigcup_{n \geq 0} \Phi^n(\emptyset),$$

with stabilization after at most $|\Omega_{\mathcal{P}}|$ strict increases. $\square$

Theorem 10 (Minimality of the least fixpoint). *Let $S^* = \text{lfp}(\Phi_{\mathcal{P}})$ and let $S \subseteq \Omega_{\mathcal{P}}$ satisfy $\Phi_{\mathcal{P}}(S) \subseteq S$ (i.e., S is a pre-fixpoint, or closed under one-step consequences). Then, $S^* \subseteq S$.*

PROOF. Let $S_0 = \emptyset$ and $S_{n+1} = \Phi_{\mathcal{P}}(S_n)$. We show by induction that $S_n \subseteq S$ for all n . Base case: $S_0 = \emptyset \subseteq S$. Inductive step: assume $S_n \subseteq S$. By monotonicity, $\Phi_{\mathcal{P}}(S_n) \subseteq \Phi_{\mathcal{P}}(S)$. By assumption $\Phi_{\mathcal{P}}(S) \subseteq S$. Therefore, $S_{n+1} = \Phi_{\mathcal{P}}(S_n) \subseteq S$. Taking unions over n yields $S^* = \bigcup_n S_n \subseteq S$. $\square$

B.5 Stratified semantics: well-definedness, uniqueness, and order-independence

Theorem 11 (Well-definedness and uniqueness of stratified semantics). *If $\mathcal{P}$ is stratified and $\Omega_{\mathcal{P}}$ is finite, then the stratified semantics $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^{\text{strat}}$ is well-defined and unique.*

PROOF. Let the stratification be $\sigma : \text{gen}(\mathcal{P}) \rightarrow \mathbb{N}$, and let $S_{-1} = \emptyset$ and for each $i \geq 0$:

$$F_i(S) := \Phi_{\mathcal{P} \leq i}(S \cup S_{i-1}), \quad S_i := \text{lfp}(F_i),$$

where $\mathcal{P} \leq i$ denotes the restriction of $\mathcal{P}$ to GenOp occurrences in strata at most i , as defined in the main text.

We prove monotonicity of each F_i , which ensures existence and uniqueness of S_i . Let $S \subseteq S'$. Then, $S \cup S_{i-1} \subseteq S' \cup S_{i-1}$. We claim $\Phi_{\mathcal{P} \leq i}$ is monotone in its context argument when all negative

dependencies of generators in strata $\leq i$ point strictly below i and are, therefore, evaluated against a fixed lower-stratum supply S_{i-1} . Stratification guarantees that any occurrence of an output variable of a stratum $\leq i$ generator under a negative operator can only depend on generators in strictly lower strata, hence, is already fixed when computing stratum i .

Thus, as the varying part S grows, no negative test that depends on generated variables can change from true to false. Therefore, Lemma 4 applies to the operator $S \mapsto \Phi_{\mathcal{P} \leq i}(S \cup S_{i-1})$, producing $F_i(S) \subseteq F_i(S')$.

Hence, F_i is monotone on $(2^{\Omega_{\mathcal{P}}}, \subseteq)$; therefore, by Theorem 9 it has a unique least fixpoint S_i . Since $\Omega_{\mathcal{P}}$ is finite, each stratum stabilizes in finitely many steps. The final stratum value S_m (with $m = \max \sigma$) is, therefore, well-defined and unique. $\square$

Theorem 12 (Order-independence within strata). *Let $\mathcal{P}$ be stratified and let $i \geq 0$. Then, the computed stratum value $S_i = \text{lfp}(F_i)$ is independent of any operational evaluation order of GenOp occurrences within stratum i , provided that: (i) lower strata $< i$ are fully computed first, and (ii) within stratum i , evaluation is fair in the sense that any enabled GenOp occurrence is eventually applied.*

PROOF. Within stratum i , semantics is defined denotationally as the least fixpoint of the monotone operator F_i on the complete lattice $2^{\Omega_{\mathcal{P}}}$.

Let $(T_n)_{n \geq 0}$ be the Kleene sequence for F_i : $T_0 = \emptyset$, $T_{n+1} = F_i(T_n)$. Then, $S_i = \bigcup_n T_n$ by Theorem 9.

Any fair operational schedule that repeatedly applies enabled GenOp occurrences and closes under the algebra corresponds to generating an ascending chain $(U_\alpha)_\alpha$ such that: (1) $U_0 = \emptyset$, (2) $U_\alpha \subseteq U_{\alpha+1} \subseteq F_i(U_\alpha)$, and (3) the limit $U_\infty := \bigcup_\alpha U_\alpha$ is a fixpoint of F_i (fairness ensures closure under all F_i -consequences).

Since T_n is the least chain closed under full application of F_i , we have by induction $T_n \subseteq U_\infty$ for all n , hence $S_i = \bigcup_n T_n \subseteq U_\infty$. For the reverse inclusion, we prove $U_\alpha \subseteq S_i$ by induction on the schedule. The base case is $U_0 = \emptyset \subseteq S_i$. If $U_\alpha \subseteq S_i$, then monotonicity and $F_i(S_i) = S_i$ give $U_{\alpha+1} \subseteq F_i(U_\alpha) \subseteq F_i(S_i) = S_i$; at limit stages, inclusion is preserved by unions. Hence, $U_\infty \subseteq S_i$. Therefore, $U_\infty = S_i$, showing independence of the operational order. $\square$

B.6 Well-founded semantics: convergence and uniqueness

Theorem 13 (Convergence and uniqueness of well-founded semantics). *If $\text{Cand}(\mathcal{P})$ is finite, then the alternating fixpoint sequence (I_n^+, I_n^-) from Definition 21 stabilizes after finitely many steps at a pair (I^+, I^-) , and the induced three-valued interpretation is unique and is the well-founded model relative to $\text{Cand}(\mathcal{P})$ and the fixed reduct family.*

PROOF. Let $C := \text{Cand}(\mathcal{P})$ and

$$\Gamma_{\mathcal{P}}(J) := \text{lfp}(S \mapsto \Phi_{\mathcal{P}}^J(S)) \quad (J \subseteq C).$$

By the reduct conditions in Section 4.5, each $\Phi_{\mathcal{P}}^J$ is monotone in S , and for $J \subseteq J'$ and every $S \subseteq C$,

$$\Phi_{\mathcal{P}}^{J'}(S) \subseteq \Phi_{\mathcal{P}}^J(S).$$

Induction on the corresponding Kleene sequences therefore gives

$$J \subseteq J' \implies \Gamma_{\mathcal{P}}(J') \subseteq \Gamma_{\mathcal{P}}(J). \quad (\text{D})$$

Thus, $\Gamma_{\mathcal{P}}$ is antitone.

Let $L_n := I_n^+$ and $U_n := C \setminus I_n^-$. Definition 21 is equivalent to

$$L_{n+1} = \Gamma_{\mathcal{P}}(U_n) \quad \text{and} \quad U_{n+1} = \Gamma_{\mathcal{P}}(L_{n+1}), \quad (\text{E})$$

with $L_0 = \emptyset$ and $U_0 = C$. We prove simultaneously

$$L_n \subseteq L_{n+1} \quad \text{and} \quad U_{n+1} \subseteq U_n \quad (\text{F})$$

for all n . For $n = 0$, $L_0 = \emptyset \subseteq L_1$ and $U_1 \subseteq C = U_0$. Assume $U_n \subseteq U_{n-1}$. Antitonicity (D) gives

$$L_n = \Gamma_{\mathcal{P}}(U_{n-1}) \subseteq \Gamma_{\mathcal{P}}(U_n) = L_{n+1}.$$

Applying (D) again yields

$$U_{n+1} = \Gamma_{\mathcal{P}}(L_{n+1}) \subseteq \Gamma_{\mathcal{P}}(L_n) = U_n.$$

The induction proves (F).

We also have $L_n \subseteq U_n$ for every n . The case $n = 0$ is immediate. Since $L_1 = \Gamma_{\mathcal{P}}(C) \subseteq C$ and $\Gamma_{\mathcal{P}}$ is antitone,

$$L_1 = \Gamma_{\mathcal{P}}(C) \subseteq \Gamma_{\mathcal{P}}(L_1) = U_1.$$

For $n \geq 1$, assume $L_n \subseteq U_n$. Antitonicity gives

$$L_{n+1} = \Gamma_{\mathcal{P}}(U_n) \subseteq \Gamma_{\mathcal{P}}(L_n) = U_n.$$

Since $L_{n+1} \subseteq U_n$, antitonicity also gives

$$L_{n+1} = \Gamma_{\mathcal{P}}(U_n) \subseteq \Gamma_{\mathcal{P}}(L_{n+1}) = U_{n+1}.$$

Thus, $I_n^+ \cap I_n^- = \emptyset$ at every stage.

Because C is finite, the ascending chain $(L_n)_n$ and the descending chain $(U_n)_n$ stabilize. Let L and U be their stable values. Equation (E) gives

$$L = \Gamma_{\mathcal{P}}(U) \quad \text{and} \quad U = \Gamma_{\mathcal{P}}(L).$$

Hence, $I^+ := L$ and $I^- := C \setminus U$ satisfy the alternating-fixpoint equations in Definition 21. The sequence is determined uniquely by $L_0 = \emptyset$, $U_0 = C$, and (E); therefore, the stable pair is unique. By Definition 21, the stable pair is the well-founded model relative to C and the fixed reduct family. $\square$

C Algebraic Equivalences and Rewrite Rules

We state all equivalences with respect to the denotational semantics $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}$ of Section 3 over a fixed RDF graph $\mathcal{G}$.

We assume the underlying string similarity $\text{Str}_{\approx} : \Sigma^* \times \Sigma^* \rightarrow [0, 1]$ is *symmetric*, i.e., $\text{Str}_{\approx}(s, s') = \text{Str}_{\approx}(s', s)$ for all $s, s' \in \Sigma^*$.

Lemma 5 (Symmetry of typed-term similarity). *For all typed terms $\alpha, \beta \in (\mathcal{T} \times \{\text{rdf}\}) \cup (\mathcal{T}_{\text{gen}} \times \{\text{gen}\})$, we have $\text{Sim}(\alpha, \beta) = \text{Sim}(\beta, \alpha)$, where $\text{Sim}(\cdot, \cdot)$ is defined with $\text{Str}_{\approx}$ in Section 3.*

PROOF. Let $\alpha = (t, \tau)$ and $\beta = (t', \tau')$. If $\tau = \tau' = \text{rdf}$, then

$$\text{Sim}(\alpha, \beta) = \begin{cases} 1 & t = t', \\ 0 & t \neq t', \end{cases}$$

and symmetry is immediate. If $(\tau, \tau') = (\text{rdf}, \text{gen})$, then

$$\text{Sim}(\alpha, \beta) = \text{Str}_{\approx}(\text{Canon}(t), \text{Lex}(t')),$$

$$\text{Sim}(\beta, \alpha) = \text{Str}_{\approx}(\text{Lex}(t'), \text{Canon}(t)),$$

which are equal by symmetry of $\text{Str}_{\approx}$. The remaining two cases are analogous. $\square$

Lemma 6 (Symmetry of canonical compatibility). *For every variable $v \in \mathcal{V}$ and typed terms α, β ,*

$$\text{Compat}_v(\alpha, \beta) = \top \quad \text{iff} \quad \text{Compat}_v(\beta, \alpha) = \top.$$

PROOF. By definition, $\text{Compat}_v(\alpha, \beta) = \top$ iff $\text{rep}_v(\alpha) = \text{rep}_v(\beta)$, which is symmetric. $\square$

Lemma 7 (Symmetry of mapping compatibility). *For all typed solution mappings μ_1, μ_2 ,*

$$\mu_1 \sim \mu_2 \quad \text{iff} \quad \mu_2 \sim \mu_1.$$

PROOF. By Definition 9, $\mu_1 \sim \mu_2$ holds iff for every $v \in \text{dom}_{\mu_1} \cap \text{dom}_{\mu_2}$ we have $\text{rep}_v(\mu_1(v)) = \text{rep}_v(\mu_2(v))$. The equality is symmetric in μ_1, μ_2 . $\square$

Lemma 8 (Canonical union preserves bindings outside overlaps). *If $\mu_1 \sim \mu_2$, then:*

- (1) *for every $v \in \text{dom}_{\mu_1} \setminus \text{dom}_{\mu_2}$, $(\mu_1 \uplus \mu_2)(v) = \mu_1(v)$;*
- (2) *for every $v \in \text{dom}_{\mu_2} \setminus \text{dom}_{\mu_1}$, $(\mu_1 \uplus \mu_2)(v) = \mu_2(v)$;*
- (3) *for every $v \in \text{dom}_{\mu_1} \cap \text{dom}_{\mu_2}$, $(\mu_1 \uplus \mu_2)(v) = \text{rep}_v(\mu_1(v)) = \text{rep}_v(\mu_2(v))$.*

PROOF. Each clause is the corresponding case of Definition 9. $\square$

Lemma 9 (Canonical union is commutative). *If $\mu_1 \sim \mu_2$, then $\mu_1 \uplus \mu_2 = \mu_2 \uplus \mu_1$.*

PROOF. By Lemma 8, both unions agree on variables outside the overlap. On the overlap $\text{dom}_{\mu_1} \cap \text{dom}_{\mu_2}$, both yield the representative $\text{rep}_v(\mu_1(v)) = \text{rep}_v(\mu_2(v))$. Hence, the resulting partial functions are equal. $\square$

C.0.1 Join. From Section 3:

$$[\mathcal{P}_1 \text{ Join } \mathcal{P}_2]_{\mathcal{G}} = \{\mu_1 \uplus \mu_2 \mid \mu_1 \in [\mathcal{P}_1]_{\mathcal{G}}, \mu_2 \in [\mathcal{P}_2]_{\mathcal{G}}, \mu_1 \sim \mu_2\}.$$

Proposition 7. *For all patterns $\mathcal{P}_1, \mathcal{P}_2$,*

$$[\mathcal{P}_1 \text{ Join } \mathcal{P}_2]_{\mathcal{G}} = [\mathcal{P}_2 \text{ Join } \mathcal{P}_1]_{\mathcal{G}}.$$

PROOF. Let $\mu \in [\mathcal{P}_1 \text{ Join } \mathcal{P}_2]_{\mathcal{G}}$. Then, $\mu = \mu_1 \uplus \mu_2$ with $\mu_i \in [\mathcal{P}_i]_{\mathcal{G}}$ and $\mu_1 \sim \mu_2$. By Lemma 7, $\mu_2 \sim \mu_1$, and by Lemma 9, $\mu = \mu_2 \uplus \mu_1 \in [\mathcal{P}_2 \text{ Join } \mathcal{P}_1]_{\mathcal{G}}$. The reverse inclusion is symmetric. $\square$

Proposition 8. *For all patterns $\mathcal{P}_1, \mathcal{P}_2, \mathcal{P}_3$,*

$$[(\mathcal{P}_1 \text{ Join } \mathcal{P}_2) \text{ Join } \mathcal{P}_3]_{\mathcal{G}} = [\mathcal{P}_1 \text{ Join } (\mathcal{P}_2 \text{ Join } \mathcal{P}_3)]_{\mathcal{G}}.$$

PROOF. We prove $\subseteq$; the reverse direction is analogous.

Let

$$\mu \in [(\mathcal{P}_1 \text{ Join } \mathcal{P}_2) \text{ Join } \mathcal{P}_3]_{\mathcal{G}}.$$

Then, there are $\mu_{12} \in [\mathcal{P}_1 \text{ Join } \mathcal{P}_2]_{\mathcal{G}}$ and $\mu_3 \in [\mathcal{P}_3]_{\mathcal{G}}$ such that $\mu = \mu_{12} \uplus \mu_3$ and $\mu_{12} \sim \mu_3$. Expanding the definition of μ_{12} gives $\mu_i \in [\mathcal{P}_i]_{\mathcal{G}}$ for $i \in \{1, 2\}$ with $\mu_{12} = \mu_1 \uplus \mu_2$ and $\mu_1 \sim \mu_2$.

Claim 1: $\mu_2 \sim \mu_3$. Let $v \in \text{dom}_{\mu_2} \cap \text{dom}_{\mu_3}$. Then, $v \in \text{dom}_{\mu_{12}} \cap \text{dom}_{\mu_3}$, and $\mu_{12} \sim \mu_3$ gives $\text{rep}_v(\mu_{12}(v)) = \text{rep}_v(\mu_3(v))$. By Lemma 8, $\mu_{12}(v)$ is either $\mu_2(v)$ or $\text{rep}_v(\mu_2(v))$. Hence, $\text{rep}_v(\mu_{12}(v)) = \text{rep}_v(\mu_2(v))$; therefore, $\text{rep}_v(\mu_2(v)) = \text{rep}_v(\mu_3(v))$. Thus, $\mu_2 \sim \mu_3$.

Let $\mu_{23} := \mu_2 \uplus \mu_3$. Then, $\mu_{23} \in [\mathcal{P}_2 \text{ Join } \mathcal{P}_3]_{\mathcal{G}}$.

Claim 2: $\mu_1 \sim \mu_{23}$ and $\mu = \mu_1 \uplus \mu_{23}$. Let $v \in \text{dom}_{\mu_1} \cap \text{dom}_{\mu_{23}}$. If $v \in \text{dom}_{\mu_2}$, then $\mu_1 \sim \mu_2$ gives $\text{rep}_v(\mu_1(v)) = \text{rep}_v(\mu_2(v))$. Since $\mu_{23}(v)$ is either $\mu_2(v)$ or $\text{rep}_v(\mu_2(v))$, we obtain $\text{rep}_v(\mu_1(v)) =$

$\text{rep}_v(\mu_{23}(v))$. If $v \in \text{dom}_{\mu_3} \setminus \text{dom}_{\mu_2}$, then $v \in \text{dom}_{\mu_{12}} \cap \text{dom}_{\mu_3}$ and $\mu_{12} \sim \mu_3$. Lemma 8 gives $\mu_{12}(v) = \mu_1(v)$ and $\mu_{23}(v) = \mu_3(v)$, hence again $\text{rep}_v(\mu_1(v)) = \text{rep}_v(\mu_{23}(v))$. Therefore, $\mu_1 \sim \mu_{23}$.

For equality, both $(\mu_1 \uplus \mu_2) \uplus \mu_3$ and $\mu_1 \uplus (\mu_2 \uplus \mu_3)$ are defined (by the compatibilities shown) and, for each variable v in the union domain, both evaluate to: (i) the unique binding outside the overlap when v occurs in exactly one of μ_1, μ_2, μ_3 , or (ii) $\text{rep}_v(\cdot)$ of the shared equivalence class when v occurs in two or three of them. Hence, $(\mu_1 \uplus \mu_2) \uplus \mu_3 = \mu_1 \uplus (\mu_2 \uplus \mu_3)$.

Thus, $\mu = \mu_{12} \uplus \mu_3 = (\mu_1 \uplus \mu_2) \uplus \mu_3 = \mu_1 \uplus (\mu_2 \uplus \mu_3) = \mu_1 \uplus \mu_{23}$, and therefore $\mu \in [\mathcal{P}_1 \text{ Join } (\mathcal{P}_2 \text{ Join } \mathcal{P}_3)]_{\mathcal{G}}$. $\square$

Proposition 9. *For all patterns $\mathcal{P}_1, \mathcal{P}_2, \mathcal{P}_3$,*

$$[(\mathcal{P}_1 \text{ Union } \mathcal{P}_2) \text{ Join } \mathcal{P}_3]_{\mathcal{G}} = [(\mathcal{P}_1 \text{ Join } \mathcal{P}_3) \text{ Union } (\mathcal{P}_2 \text{ Join } \mathcal{P}_3)]_{\mathcal{G}}.$$

PROOF. ($\subseteq$) Let $\mu \in [(\mathcal{P}_1 \text{ Union } \mathcal{P}_2) \text{ Join } \mathcal{P}_3]_{\mathcal{G}}$. Then, $\mu = \mu_{12} \uplus \mu_3$ with $\mu_3 \in [\mathcal{P}_3]_{\mathcal{G}}$, $\mu_{12} \in [(\mathcal{P}_1 \text{ Union } \mathcal{P}_2)]_{\mathcal{G}}$, and $\mu_{12} \sim \mu_3$. By the definition of union, $\mu_{12} \in [\mathcal{P}_1]_{\mathcal{G}}$ or $\mu_{12} \in [\mathcal{P}_2]_{\mathcal{G}}$. In the first case, $\mu \in [\mathcal{P}_1 \text{ Join } \mathcal{P}_3]_{\mathcal{G}}$; in the second, $\mu \in [\mathcal{P}_2 \text{ Join } \mathcal{P}_3]_{\mathcal{G}}$. Thus, μ is in the union on the right-hand side.

($\supseteq$) If $\mu \in [\mathcal{P}_1 \text{ Join } \mathcal{P}_3]_{\mathcal{G}}$, then $\mu = \mu_1 \uplus \mu_3$ with $\mu_1 \in [\mathcal{P}_1]_{\mathcal{G}}$. Since $[\mathcal{P}_1]_{\mathcal{G}} \subseteq [(\mathcal{P}_1 \text{ Union } \mathcal{P}_2)]_{\mathcal{G}}$, we have $\mu \in [(\mathcal{P}_1 \text{ Union } \mathcal{P}_2) \text{ Join } \mathcal{P}_3]_{\mathcal{G}}$. The argument for $\mathcal{P}_2 \text{ Join } \mathcal{P}_3$ is identical. $\square$

C.0.2 Filter laws (grounded in ε -semantics).

Proposition 10 (Filter decomposition). *For all patterns P and filter formulas $\mathcal{F}_1, \mathcal{F}_2$,*

$$[\text{Filter } \mathcal{F}_1 \wedge \mathcal{F}_2(P)]_{\mathcal{G}} = [\text{Filter } \mathcal{F}_1(\text{Filter } \mathcal{F}_2(P))]_{\mathcal{G}}.$$

PROOF. By definition, $[\text{Filter } F(P)]_{\mathcal{G}} = \{\mu \in [\mathcal{P}]_{\mathcal{G}} \mid F^\mu = \top\}$. By the inductive filter semantics, $(\mathcal{F}_1 \wedge \mathcal{F}_2)^\mu = \mathcal{F}_1^\mu \wedge \mathcal{F}_2^\mu$, and in particular $(\mathcal{F}_1 \wedge \mathcal{F}_2)^\mu = \top$ iff $\mathcal{F}_1^\mu = \top$ and $\mathcal{F}_2^\mu = \top$. Thus, both sides select exactly the same mappings from $[\mathcal{P}]_{\mathcal{G}}$. $\square$

Lemma 10 (Filter invariance under representative normalization). *Let μ be a typed solution mapping and let $\widehat{\mu}(v) = \text{rep}_v(\mu(v))$ for every $v \in \text{dom}_\mu$. If a filter formula $\mathcal{F}$ contains no atom of the form $(v = v')$, then $\mathcal{F}^\mu = \mathcal{F}^{\widehat{\mu}}$.*

PROOF. We use structural induction on $\mathcal{F}$.

If $\mathcal{F} = \text{bound}(v)$, then $\text{dom}_{\widehat{\mu}} = \text{dom}_\mu$; therefore, the two truth values are equal.

Let $\mathcal{F} = (v = c)$. If $v \notin \text{dom}_\mu$, both evaluations equal ε . Assume $v \in \text{dom}_\mu$. Since $\widehat{\mu}(v) = \text{rep}_v(\mu(v))$ and rep_v is idempotent,

$$\text{rep}_v(\widehat{\mu}(v)) = \text{rep}_v(\mu(v)).$$

Therefore,

$$\begin{aligned} \text{rep}_v(\mu(v)) = \text{rep}_v((c, \text{rdf})) &\iff \text{rep}_v(\widehat{\mu}(v)) = \text{rep}_v((c, \text{rdf})) \\ &\iff (v = c)^{\widehat{\mu}} = \top. \end{aligned}$$

For bound v , the first representative equality is equivalent to $(v = c)^\mu = \top$ by the filter clause; exact RDF equality is a special case. Hence, $(v = c)^\mu = (v = c)^{\widehat{\mu}}$.

The atom $(v = v')$ is excluded by hypothesis. For $\neg \mathcal{F}_1$, the induction hypothesis and the three-valued negation clause give equality. For $\mathcal{F}_1 \wedge \mathcal{F}_2$, the induction hypotheses and the conjunction clause

give equality. The cases exhaust the filter grammar under the stated restriction. $\square$

Proposition 11 (Filter pushdown over join (safe form)). *Let $\mathcal{P}_1, \mathcal{P}_2$ be patterns containing no GenOp occurrence, and let $\mathcal{F}$ be a filter formula. Assume $\text{var}(\mathcal{F}) \subseteq \text{dom}_{\mu_1}$ for every $\mu_1 \in \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}}$. Then,*

$$\llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P}_1 \text{ Join } \mathcal{P}_2) \rrbracket_{\mathcal{G}} = \llbracket (\text{Filter}_{\mathcal{F}}(\mathcal{P}_1)) \text{ Join } \mathcal{P}_2 \rrbracket_{\mathcal{G}}.$$

Symmetrically, the corresponding equality holds for pushdown to $\mathcal{P}_2$ if $\text{var}(\mathcal{F}) \subseteq \text{dom}_{\mu_2}$ for every $\mu_2 \in \llbracket \mathcal{P}_2 \rrbracket_{\mathcal{G}}$.

PROOF. We prove the first equality. Let $\mu_1 \in \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}}$ and $\mu_2 \in \llbracket \mathcal{P}_2 \rrbracket_{\mathcal{G}}$ satisfy $\mu_1 \sim \mu_2$, and let $\mu := \mu_1 \uplus \mu_2$. Because $\mathcal{P}_1$ and $\mathcal{P}_2$ contain no GenOp occurrence, all bindings in μ_1 and μ_2 are RDF-typed. Definition 9 therefore gives

$$\forall v \in \text{dom}_{\mu_1} \cap \text{dom}_{\mu_2} : \mu_1(v) = \mu_2(v),$$

and μ agrees with μ_1 on dom_{μ_1} . Since $\text{var}(\mathcal{F}) \subseteq \text{dom}_{\mu_1}$, we obtain

$$\mu|_{\text{var}(\mathcal{F})} = (\mu_1)|_{\text{var}(\mathcal{F})}. \quad (\text{G})$$

We derive $\mathcal{F}^{\mu} = \mathcal{F}^{\mu_1}$ by structural induction on $\mathcal{F}$. For $\text{bound}(v)$, both mappings bind v . For $(v = c)$, equation (G) gives $\mu(v) = \mu_1(v)$. For $(v = v')$, it gives $\mu(v) = \mu_1(v)$ and $\mu(v') = \mu_1(v')$. The clauses for $\neg \mathcal{F}_1$ and $\mathcal{F}_1 \wedge \mathcal{F}_2$ follow from the corresponding induction hypotheses and the filter semantics. Hence,

$$\mathcal{F}^{\mu} = \mathcal{F}^{\mu_1}. \quad (\text{H})$$

Let $A_i := \llbracket \mathcal{P}_i \rrbracket_{\mathcal{G}}$. By the join and filter clauses and (H),

$$\begin{aligned} & \llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P}_1 \text{ Join } \mathcal{P}_2) \rrbracket_{\mathcal{G}} \\ &= \left\{ \mu_1 \uplus \mu_2 \mid \begin{array}{l} \mu_1 \in A_1, \mu_2 \in A_2, \mu_1 \sim \mu_2, \\ \mathcal{F}^{\mu_1} = \top \end{array} \right\} \\ &= \left\{ \mu_1 \uplus \mu_2 \mid \begin{array}{l} \mu_1 \in \llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P}_1) \rrbracket_{\mathcal{G}}, \\ \mu_2 \in A_2, \mu_1 \sim \mu_2 \end{array} \right\} \\ &= \llbracket (\text{Filter}_{\mathcal{F}}(\mathcal{P}_1)) \text{ Join } \mathcal{P}_2 \rrbracket_{\mathcal{G}}. \end{aligned}$$

The second equality is symmetric. $\square$

C.0.3 Minus.

Proposition 12. *If $\text{var}(\mathcal{P}_1) \cap \text{var}(\mathcal{P}_2) = \emptyset$, then*

$$\llbracket \mathcal{P}_1 \text{ Minus } \mathcal{P}_2 \rrbracket_{\mathcal{G}} = \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}}.$$

PROOF. By definition, $\llbracket \mathcal{P}_1 \text{ Minus } \mathcal{P}_2 \rrbracket_{\mathcal{G}}$ contains exactly those $\mu_1 \in \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}}$ such that for all $\mu_2 \in \llbracket \mathcal{P}_2 \rrbracket_{\mathcal{G}}$, either $\mu_1 \approx \mu_2$ or $\text{dom}_{\mu_1} \cap \text{dom}_{\mu_2} = \emptyset$. If $\text{var}(\mathcal{P}_1) \cap \text{var}(\mathcal{P}_2) = \emptyset$, then for all such μ_1, μ_2 we have $\text{dom}_{\mu_1} \cap \text{dom}_{\mu_2} = \emptyset$; therefore, no μ_1 is removed. Thus, $\llbracket \mathcal{P}_1 \text{ Minus } \mathcal{P}_2 \rrbracket_{\mathcal{G}} = \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}}$. $\square$

C.1 GenOp-Specific Rewriting Rules

The rewriting rules depend on the *context-sensitive* evaluation of GenOp patterns (Section 3) and are therefore sound only under explicit side conditions.

Throughout, let $\mathcal{G}$ be an RDF graph and $\mathcal{P}$ a well-formed pattern.

C.1.1 Acyclic setting and stagewise (single-pass) evaluation. Assume the dependency graph $\vec{\mathcal{D}}_{\mathcal{P}} = (\text{gen}(\mathcal{P}), \rightarrow_{\mathcal{P}})$ is acyclic, and let $\gamma = (g_1, \dots, g_m)$ be a topological order of $\text{gen}(\mathcal{P})$. For each $k \in \{0, \dots, m\}$, Appendix B.1 defines $\mathcal{P}_Y^{[k]}$ as the pattern obtained from $\mathcal{P}$ by disabling all GenOp occurrences not among $\{g_1, \dots, g_k\}$. The stagewise sequence is

$$S_Y^{(0)} := \llbracket \mathcal{P}_Y^{[0]} \rrbracket_{\mathcal{G}}^{\emptyset} \quad \text{and} \quad S_Y^{(k)} := \llbracket \mathcal{P}_Y^{[k]} \rrbracket_{\mathcal{G}}^{S_Y^{(k-1)}} \quad (1 \leq k \leq m).$$

At stage k , we evaluate only generators among $\{g_1, \dots, g_k\}$, using as context the supply accumulated at stage $k-1$.

For a GenOp occurrence $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$ and a supply S , we recall the *context restriction* operator:

$$S|_g := \{\mu \in S \mid X = \text{input}(g) \subseteq \text{dom}_{\mu}\}.$$

C.1.2 Context Independence of Base-Mode GenOp.

Proposition 13 (Base-mode context independence). *Let $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$ be a GenOp pattern with $\text{input}(g) = X = \emptyset$. Assume the base-mode convention of Section 3 that whenever $X = \emptyset$, the GenOp is evaluated with context set $\{\emptyset\}$. Then, for every supply $S \subseteq \Omega_{\mathcal{P}}$,*

$$S_g = \{\emptyset\} \quad \text{and} \quad \llbracket g \rrbracket_{\mathcal{G}}^{S_g} = \llbracket g \rrbracket_{\mathcal{G}}^{\{\emptyset\}},$$

i.e., the denotation of g is independent of S .

PROOF. The claim follows from the $X = \emptyset$ clause in the definition of S_g . $\square$

Corollary 2 (Hoisting base-mode GenOp). *Let*

$$g = \text{GenOp}(\pi(\emptyset), Y, \mathcal{M}) \quad \text{and} \quad \Omega_g = \llbracket g \rrbracket_{\mathcal{G}}^{\{\emptyset\}}.$$

Let $\mathcal{P}$ contain no GenOp occurrence. For every RDF graph $\mathcal{G}$ and context supplies S, S' ,

$$\begin{aligned} \llbracket \mathcal{P} \text{ Join } g \rrbracket_{\mathcal{G}}^S &= \llbracket \mathcal{P} \text{ Join } g \rrbracket_{\mathcal{G}}^{S'} \\ &= \llbracket \mathcal{P} \text{ Join } \Omega_g \rrbracket_{\mathcal{G}}. \end{aligned}$$

where Ω_g is treated extensionally as a relation (set of typed solution mappings).

PROOF. Let S be an arbitrary context supply. *Step 1 (base-mode context-independence of g).* Since $g = \text{GenOp}(\pi(\emptyset), Y, \mathcal{M})$ is base-mode, Proposition 13 yields

$$\llbracket g \rrbracket_{\mathcal{G}}^{S_g} = \llbracket g \rrbracket_{\mathcal{G}}^{\{\emptyset\}} = \Omega_g. \quad (*)$$

That is, $\llbracket g \rrbracket_{\mathcal{G}}^{S_g}$ is independent of S .

Step 2 (context-independence of $\mathcal{P}$). Because $\mathcal{P}$ contains no GenOp, its evaluation does not depend on the context supply (Proposition 18). Hence, for all supplies S, S' ,

$$\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S = \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^{S'} = \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}. \quad (\dagger)$$

Step 3 (join evaluation and elimination of the context parameter). By the semantic clause for join (Section 3),

$$\llbracket \mathcal{P} \text{ Join } g \rrbracket_{\mathcal{G}}^S = \{\mu \uplus \nu \mid \mu \in \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^S, \nu \in \llbracket g \rrbracket_{\mathcal{G}}^{S_g}, \mu \sim \nu\}.$$

Substituting $(*)$ and $(\dagger)$ gives

$$\llbracket \mathcal{P} \text{ Join } g \rrbracket_{\mathcal{G}}^S = \{\mu \uplus \nu \mid \mu \in \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}, \nu \in \Omega_g, \mu \sim \nu\},$$

which is independent of S .

Therefore, $\llbracket \mathcal{P} \text{ Join } g \rrbracket_{\mathcal{G}}^S = \llbracket \mathcal{P} \text{ Join } g \rrbracket_{\mathcal{G}}^{S'}$ for all S, S' . Treating Ω_g extensionally as a relation and applying the join clause of the denotational semantics gives

$$\{\mu \uplus v \mid \mu \in \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}, v \in \Omega_g, \mu \sim v\} = \llbracket \mathcal{P} \text{ Join } \Omega_g \rrbracket_{\mathcal{G}}.$$

The two equalities prove the claim. $\square$

Proposition 14. *Let $\mathcal{P}_1, \mathcal{P}_2$ be patterns, g a GenOp, $\mathcal{G}$ an RDF graph, and S a context supply. For $i \in \{1, 2\}$, let*

$$S_i := \llbracket \mathcal{P}_i \rrbracket_{\mathcal{G}}^S \quad \text{and} \quad A_i := \llbracket g \rrbracket_{\mathcal{G}}^{S_i|g}.$$

On the right-hand side, let g_1, g_2 denote the two occurrences of g and assume $S_{g_i} = S_i|_{g_i}$ for $i \in \{1, 2\}$. Assume

$$\llbracket g \rrbracket_{\mathcal{G}}^{S_1|g \cup S_2|g} = A_1 \cup A_2$$

and, for $i \neq j$,

$$\forall \mu_i \in S_i \forall v_j \in A_j : \mu_i \approx v_j.$$

Then,

$$\llbracket (\mathcal{P}_1 \text{ Union } \mathcal{P}_2) \text{ Join } g \rrbracket_{\mathcal{G}}^S = \llbracket (\mathcal{P}_1 \text{ Join } g) \text{ Union } (\mathcal{P}_2 \text{ Join } g) \rrbracket_{\mathcal{G}}^S.$$

PROOF. The surrounding pattern of g in $(\mathcal{P}_1 \text{ Union } \mathcal{P}_2) \text{ Join } g$ supplies

$$(S_1 \cup S_2)|_g = S_1|_g \cup S_2|_g.$$

Hence, the left-hand side equals

$$\{\mu \uplus v \mid \mu \in S_1 \cup S_2, v \in A_1 \cup A_2, \mu \sim v\}. \quad (\text{H})$$

Let (μ, v) witness a mapping in (H). Choose $i, j \in \{1, 2\}$ with $\mu \in S_i$ and $v \in A_j$. If $i \neq j$, the second premise gives $\mu \approx v$, contradicting the witness condition. Therefore, $i = j$, and (H) equals

$$\bigcup_{i=1}^2 \{\mu \uplus v \mid \mu \in S_i, v \in A_i, \mu \sim v\}.$$

By the context premise for g_1, g_2 and the join and union clauses, the union equals

$$\llbracket (\mathcal{P}_1 \text{ Join } g) \text{ Union } (\mathcal{P}_2 \text{ Join } g) \rrbracket_{\mathcal{G}}^S.$$

$\square$

Proposition 15. *Let $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$ and let $\mathcal{F}$ be a filter such that $\text{var}(\mathcal{F}) \subseteq X$ and $\mathcal{F}$ contains no atom of the form $(v = v')$. Let $\mathcal{P}$ satisfy $Y \cap \text{var}(\mathcal{P}) = \emptyset$, $\text{var}(\mathcal{F}) \subseteq \text{var}(\mathcal{P})$, and*

$$\forall \mu \in \llbracket \mathcal{P} \rrbracket_{\mathcal{G}} : X \subseteq \text{dom}_{\mu}.$$

Then,

$$\llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P}) \text{ Join } g \rrbracket_{\mathcal{G}} = \llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P} \text{ Join } g) \rrbracket_{\mathcal{G}}.$$

PROOF. Let

$$S := \llbracket \mathcal{P} \rrbracket_{\mathcal{G}} \quad \text{and} \quad S^+ := \{\mu \in S \mid \mathcal{F}^{\mu} = \top\}.$$

Deleting the occurrence g from either side and applying the pruning rules yields $\text{Filter}_{\mathcal{F}}(\mathcal{P})$. Hence, by the definition of the actual occurrence context and the domain premise, $S_g = S^+|_g = S^+$ on both sides. For any $T \subseteq S$, Definition 10 gives

$$\llbracket g \rrbracket_{\mathcal{G}}^T = \bigcup_{\eta \in T} \llbracket g \rrbracket_{\mathcal{G}}^{\{\eta\}}. \quad (\text{I})$$

Let $\mu \in S$, $\eta \in S^+$, and let $\theta \in \llbracket g \rrbracket_{\mathcal{G}}^{\{\eta\}}$ satisfy $\mu \sim \theta$. Then, $\theta = \eta \uplus v_Y$ for a mapping v_Y on Y . Since $Y \cap \text{var}(\mathcal{P}) = \emptyset$ and

$\text{var}(\mathcal{F}) \subseteq X \subseteq \text{var}(\mathcal{P})$, v_Y binds no variable of $\mathcal{F}$. For every $v \in \text{var}(\mathcal{F})$, $v \in X$ and $X \cap Y = \emptyset$ give $\theta(v) = \eta(v)$; compatibility of μ and θ therefore gives

$$\text{rep}_v(\mu(v)) = \text{rep}_v(\theta(v)) = \text{rep}_v(\eta(v)).$$

If $\omega := \mu \uplus \theta$, then also $\text{rep}_v(\omega(v)) = \text{rep}_v(\mu(v))$. Lemma 10 therefore yields

$$\mathcal{F}^{\omega} = \mathcal{F}^{\mu} = \mathcal{F}^{\eta} = \top. \quad (\text{J})$$

For the left-to-right inclusion, let $\omega \in \llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P}) \text{ Join } g \rrbracket_{\mathcal{G}}$. Then, $\omega = \mu \uplus \theta$ with $\mu \in S^+$, $\theta \in \llbracket g \rrbracket_{\mathcal{G}}^{S^+}$, and $\mu \sim \theta$. By (J), $\mathcal{F}^{\omega} = \top$; therefore, $\omega \in \llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P} \text{ Join } g) \rrbracket_{\mathcal{G}}$.

Conversely, let $\omega \in \llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P} \text{ Join } g) \rrbracket_{\mathcal{G}}$. Then, $\omega = \mu \uplus \theta$ with $\mu \in S$, $\theta \in \llbracket g \rrbracket_{\mathcal{G}}^{S^+}$, $\mu \sim \theta$, and $\mathcal{F}^{\omega} = \top$. By (I), choose $\eta \in S^+$ such that $\theta \in \llbracket g \rrbracket_{\mathcal{G}}^{\{\eta\}}$. Equation (J) gives $\mathcal{F}^{\mu} = \top$; therefore, $\mu \in S^+$. Hence, $\omega \in \llbracket \text{Filter}_{\mathcal{F}}(\mathcal{P}) \text{ Join } g \rrbracket_{\mathcal{G}}$. $\square$

C.1.3 Safe Reordering of GenOp with Join.

Proposition 16 (Safe join reordering around a GenOp). *Let $g = \text{GenOp}(\pi(X), Y, \mathcal{M})$ and let $\mathcal{P}_1, \mathcal{P}_2$ be patterns such that*

- (1) $X \subseteq \text{dom}_{\mu_1}$ for every $\mu_1 \in \llbracket \mathcal{P}_1 \rrbracket_{\mathcal{G}}$;
- (2) $\text{var}(\mathcal{P}_2) \cap X = \emptyset$; and
- (3) $\llbracket \mathcal{P}_2 \rrbracket_{\mathcal{G}}$ is closed under $\uplus$ for compatible mappings.

Then, for every RDF graph $\mathcal{G}$ and context supply S ,

$$\llbracket (\mathcal{P}_1 \text{ Join } \mathcal{P}_2) \text{ Join } g \rrbracket_{\mathcal{G}}^S = \llbracket (\mathcal{P}_1 \text{ Join } g) \text{ Join } \mathcal{P}_2 \rrbracket_{\mathcal{G}}^S.$$

PROOF. Let $S_{12} = \llbracket \mathcal{P}_1 \text{ Join } \mathcal{P}_2 \rrbracket_{\mathcal{G}}^S$. In both patterns, replacing the designated occurrence g by the empty pattern and applying the pruning rules yields $\mathcal{P}_1 \text{ Join } \mathcal{P}_2$. Hence, the actual context supply of g is the same on both sides, namely

$$S_g = S_{12}|_g.$$

Therefore, g has the same denotation on both sides. The required equality now follows from associativity of join (Proposition 8). $\square$

C.1.4 Order Invariance under Acyclic GenOp Dependencies.

Proposition 17. *Assume $\vec{\mathcal{D}}_{\mathcal{P}}$ is acyclic. Let γ and γ' be two topological orders of $\text{gen}(\mathcal{P})$. Then, the final stagewise results coincide:*

$$S_Y^{(m)} = S_Y'^{(m)}.$$

PROOF. We prove invariance under adjacent swaps of incomparable generators and use the connection between topological orders by such swaps.

Step 1: invariance under a single adjacent swap. Let $\gamma = (\dots, g_i, g_{i+1}, \dots)$ be a topological order such that g_i and g_{i+1} are incomparable (no edges in either direction). Let γ' be the order obtained by swapping them: $\gamma' = (\dots, g_{i+1}, g_i, \dots)$.

Incomparability means:

$$\text{out}(g_i) \cap \text{input}(g_{i+1}) = \emptyset \quad \text{and} \quad \text{out}(g_{i+1}) \cap \text{input}(g_i) = \emptyset.$$

Let S be the context supply before either generator is applied; both orders have the same supply at that stage. Because $\text{out}(g_i)$ does not occur among placeholders of g_{i+1} , evaluating g_i may add bindings to $\text{out}(g_i)$, but cannot change which mappings in S bind all of $\text{input}(g_{i+1})$, hence it cannot change $S_{g_{i+1}}$ nor any instantiated prompt for g_{i+1} . Symmetrically, evaluating g_{i+1} first cannot change

the contexts for g_i . Therefore, the sets of generated extensions contributed by g_i and g_{i+1} are the same regardless of which is applied first, and since join is associative, the supply after applying both is identical under γ and γ' .

Step 2: connection between topological orders by adjacent swaps. Any two linear extensions of a finite poset are connected by a sequence of adjacent swaps of incomparable elements. Step 1 preserves the final result at each swap.

Therefore, $S_Y^{(m)} = S_{Y'}^{(m)}$.

(The conclusion also appears as Item (3) of Theorem 8.)

□

C.2 Conservativity w.r.t. SPARQL

Proposition 18. *Let $\mathcal{P}$ be a well-formed graph pattern that contains no occurrence of GenOp. Then, for any RDF graph $\mathcal{G}$, evaluation of $\mathcal{P}$ under the extended semantics coincides with standard SPARQL evaluation:*

$$\llbracket \mathcal{P} \rrbracket_{\mathcal{G}} = (\llbracket \mathcal{P} \rrbracket_{\mathcal{G}})|_{\text{standard SPARQL}}.$$

PROOF. Let $\mathcal{G}$ be a fixed RDF graph. Since $\mathcal{P}$ contains no GenOp, the evaluation rules for $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}$ never invoke any context-dependent generator clause and, therefore, never introduce gen-typed values. Hence, every mapping $\mu \in \llbracket \mathcal{P} \rrbracket_{\mathcal{G}}$ assigns variables only to RDF-typed terms, i.e., values in $T \times \{\text{rdf}\}$.

We first show that on RDF-typed values the extended notions of compatibility and canonical union coincide with SPARQL's usual agreement-on-overlap and set-theoretic union.

Claim 1 (RDF-typed compatibility coincides with equality). Let $v \in \mathcal{V}$ and let $\alpha = (t, \text{rdf})$ and $\beta = (t', \text{rdf})$. Then,

$$\text{Compat}_v(\alpha, \beta) = \top \quad \text{iff} \quad t = t'.$$

By definition, $\text{Compat}_v(\alpha, \beta) = \top$ iff $\text{rep}_v(\alpha) = \text{rep}_v(\beta)$. For RDF-typed inputs, the typed-term similarity $\text{Sim}(\alpha, \beta)$ equals 1 exactly when $t = t'$ and otherwise 0 (Section 3). Thus, the induced $\approx_v$ -classes are singletons; therefore, $\text{rep}_v(t, \text{rdf}) = (t, \text{rdf})$ and the equivalence follows.

Claim 2 (Mapping compatibility and canonical union coincide with SPARQL). Let μ_1, μ_2 be typed solution mappings with range contained in $T \times \{\text{rdf}\}$. Then: (i) $\mu_1 \sim \mu_2$ iff μ_1 and μ_2 agree on every shared variable; and (ii) whenever $\mu_1 \sim \mu_2$, the canonical union $\mu_1 \uplus \mu_2$ coincides with the ordinary union of partial functions $\mu_1 \cup \mu_2$. For (i), we expand the definition of $\mu_1 \sim \mu_2$: for all $v \in \text{dom}_{\mu_1} \cap \text{dom}_{\mu_2}$ we require $\text{rep}_v(\mu_1(v)) = \text{rep}_v(\mu_2(v))$, which by Claim 1 is equivalent to equality of the underlying RDF terms. For (ii), by Definition 9, $\uplus$ preserves non-overlapping bindings and, on overlaps, returns the representative; by Claim 1, the representative is the same RDF-typed binding.

We now prove the proposition by structural induction on $\mathcal{P}$.

Base case (BGP). For a basic graph pattern, the extended semantics matches triples in $\mathcal{G}$ as in SPARQL and tags bound RDF terms with `rdf`. Removing the tag yields the standard SPARQL BGP solution mappings; hence, the two evaluations coincide.

Inductive steps. Assume the claim holds for strict subpatterns. For each constructor of graph patterns used in $\mathcal{P}$ (join, union, optional, minus, difference, filter, and projection), the extended semantics is defined by the standard SPARQL set-theoretic clause, except that it uses $\sim$ for join compatibility and $\uplus$ for merging compatible

mappings, and it evaluates filter atoms using canonical compatibility. On the fragment without GenOp, all intermediate mappings are RDF-typed, and by Claims 1–2, the operations coincide with SPARQL agreement on overlaps, ordinary union, and standard filter evaluation. Therefore, each constructor yields exactly the same set of solutions as standard SPARQL.

Consequently, $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}} = (\llbracket \mathcal{P} \rrbracket_{\mathcal{G}})|_{\text{standard SPARQL}}$. □

D Proofs for Executable Fixpoint Evaluation

We prove the guarantees stated for Algorithm 1: exactness on the chosen candidate set, soundness relative to the self-support target, relative completeness under candidate coverage, and termination.

D.1 Objects and assumptions

Let $\mathcal{G}$ be an RDF graph, let $\mathcal{P}$ be a graph pattern, and let $C \subseteq \text{gen}(\mathcal{P})$ be an SCC. Let X_C and Y_C be as in the main text, and let η be an outer binding with $X_C \subseteq \text{dom}(\eta)$ and $\text{dom}(\eta) \cap Y_C = \emptyset$. Let $\Omega_{\mathcal{P}}$ denote the universe of typed solution mappings used in the semantics.

Self-support. For $\mu \in \Omega_{\mathcal{P}}$, we write $\text{SS}_C(\mu)$ if $X_C \subseteq \text{dom}(\mu)$ and, for every $g = \text{GenOp}(\pi(X_g), Y_g, \mathcal{M}) \in C$, the following conditions hold. Let $\theta_g := \mu|_{X_g}$ and $v_g := \mu|_{Y_g}$. Then,

$$\text{dom}(\theta_g) = X_g, \quad \text{dom}(v_g) = Y_g, \quad \mu|_{X_g \cup Y_g} = \theta_g \uplus v_g,$$

$$\text{and } \theta_g \uplus v_g \in \llbracket g \rrbracket_{\mathcal{G}}^{\{\theta_g\}}. \quad (\star)$$

Self-support target under an outer binding. We define the self-support answer set of C under η by

$$\text{Self}_C(\eta) = \{\mu|_{Y_C} \mid \mu \in \Omega_{\mathcal{P}}, \mu \supseteq \eta, \text{SS}_C(\mu)\}. \quad (\dagger)$$

Under the finite candidate coverage and exactness assumption from Sect. 6, the self-support answer set coincides with the declarative answer projections for the component under η .

Candidate set. Algorithm 1 constructs a finite candidate set $\text{Cand}_C(\eta) \subseteq \Omega_{\mathcal{P}}$ whose elements extend η and bind the variables in Y_C . The algorithm returns

$$R_C(\eta) = \{\mu|_{Y_C} \mid \mu \in \text{Cand}_C(\eta) \wedge \text{SS}_C(\mu)\}. \quad (\ddagger)$$

Validation. Algorithm 1 tests self-support using `Validate`. We use the following semantic correctness assumption.

Assumption 1. For every $g = \text{GenOp}(\pi(X_g), Y_g, \mathcal{M}) \in C$, every typed mapping θ with $\text{dom}(\theta) = X_g$, every typed mapping v with $\text{dom}(v) = Y_g$, and every repetition parameter R ,

$$\text{Validate}(g, \theta, v; R) = \top \iff \theta \uplus v \in \llbracket g \rrbracket_{\mathcal{G}}^{\{\theta\}}.$$

Remark 3. Assumption 1 fixes the model, decoding, parsing, normalization, and repetition policy used by the validation oracle.

D.2 Exactness on the candidate set

Theorem 14. *For any finite candidate set $\text{Cand}_C(\eta)$ constructed by the algorithm, Algorithm 1 returns exactly $R_C(\eta)$ as defined in $(\ddagger)$.*

PROOF. We prove both inclusions.

$(\subseteq)$ Let ρ be returned by the algorithm. Then, the algorithm added $\rho = \mu|_{Y_C}$ for some $\mu \in \text{Cand}_C(\eta)$ whose validation test succeeded.

Thus, for every $g = \text{GenOp}(\pi(X_g), Y_g, \mathcal{M}) \in C$, with $\theta_g := \mu|_{X_g}$ and $v_g := \mu|_{Y_g}$, the algorithm checks

$$\begin{aligned} \text{dom}(v_g) &= Y_g, \quad \mu|_{X_g \cup Y_g} = \theta_g \uplus v_g, \\ &\text{and } \text{Validate}(g, \theta_g, v_g; R) = \top. \end{aligned}$$

By Assumption 1,

$$\theta_g \uplus v_g \in \llbracket g \rrbracket_{\mathcal{G}}^{\{\theta_g\}}.$$

Thus, $\text{SS}_C(\mu)$ holds by $(\star)$. Since $\mu \in \text{Cand}_C(\eta)$ and $\rho = \mu|_{Y_C}$, we obtain $\rho \in R_C(\eta)$.

$(\supseteq)$ Let $\rho \in R_C(\eta)$. By $(\ddagger)$, there exists $\mu \in \text{Cand}_C(\eta)$ such that $\rho = \mu|_{Y_C}$ and $\text{SS}_C(\mu)$. For every $g = \text{GenOp}(\pi(X_g), Y_g, \mathcal{M}) \in C$, let

$$\theta_g := \mu|_{X_g} \quad \text{and} \quad v_g := \mu|_{Y_g}.$$

By $(\star)$,

$$\text{dom}(v_g) = Y_g, \quad \mu|_{X_g \cup Y_g} = \theta_g \uplus v_g, \quad \text{and } \theta_g \uplus v_g \in \llbracket g \rrbracket_{\mathcal{G}}^{\{\theta_g\}}.$$

Assumption 1 gives

$$\text{Validate}(g, \theta_g, v_g; R) = \top.$$

Therefore, the validation test succeeds when the algorithm processes μ , and the algorithm adds $\mu|_{Y_C} = \rho$ to the output. Hence, ρ is returned.

The two inclusions show that the returned set is exactly $R_C(\eta)$. $\square$

Self-support target. We define the self-support answer set of C under η by

$$\text{Self}_C(\eta) = \{\mu|_{Y_C} \mid \mu \in \Omega_{\mathcal{P}}, \mu \supseteq \eta, \text{SS}_C(\mu)\}. \quad (2)$$

Under the finite candidate coverage and exactness assumption from Sect. 6, the self-support answer set coincides with the declarative answer projections for the component under η .

D.3 Soundness

Theorem 15 (Soundness relative to the self-support target). *For every outer binding η ,*

$$R_C(\eta) \subseteq \text{Self}_C(\eta).$$

PROOF. Let $\rho \in R_C(\eta)$. By the definition of the returned set in $(\ddagger)$, there exists a candidate $\mu \in \text{Cand}_C(\eta)$ such that

$$\rho = \mu|_{Y_C} \quad \text{and} \quad \text{SS}_C(\mu).$$

The candidate set $\text{Cand}_C(\eta)$ is constructed from the outer binding η by assigning values to the variables generated inside C . Hence, every candidate in $\text{Cand}_C(\eta)$ extends η , and therefore

$$\mu \supseteq \eta.$$

Combining the conditions

$$\rho = \mu|_{Y_C}, \quad \mu \supseteq \eta \quad \text{and} \quad \text{SS}_C(\mu),$$

we obtain the membership condition defining $\text{Self}_C(\eta)$ in $(\dagger)$. Thus,

$$\rho \in \text{Self}_C(\eta).$$

Since ρ was arbitrary, $R_C(\eta) \subseteq \text{Self}_C(\eta)$, as in (2). The inclusion proves soundness relative to the self-support target. $\square$

D.4 Relative completeness

Theorem 16 (Relative completeness under candidate coverage). *Assume that the candidate set covers all self-supporting extensions of η :*

$$\forall \mu \in \Omega_{\mathcal{P}} : (\mu \supseteq \eta \wedge \text{SS}_C(\mu)) \Rightarrow \mu \in \text{Cand}_C(\eta). \quad (\clubsuit)$$

Then, $R_C(\eta) = \text{Self}_C(\eta)$.

PROOF. We prove both inclusions.

$(\subseteq)$ By Theorem 15, every returned projection belongs to the self-support target. Hence,

$$R_C(\eta) \subseteq \text{Self}_C(\eta).$$

$(\supseteq)$ Let $\rho \in \text{Self}_C(\eta)$. By the definition of $\text{Self}_C(\eta)$ in $(\dagger)$, there exists a mapping $\mu \in \Omega_{\mathcal{P}}$ such that

$$\rho = \mu|_{Y_C}, \quad \mu \supseteq \eta \quad \text{and} \quad \text{SS}_C(\mu).$$

The coverage assumption $(\clubsuit)$ applies to μ , which extends η and satisfies $\text{SS}_C(\mu)$. Therefore,

$$\mu \in \text{Cand}_C(\eta).$$

We have $\mu \in \text{Cand}_C(\eta)$ and $\text{SS}_C(\mu)$. By the definition of $R_C(\eta)$ in $(\ddagger)$, the projection $\mu|_{Y_C}$ belongs to $R_C(\eta)$. Since $\rho = \mu|_{Y_C}$, we obtain

$$\rho \in R_C(\eta).$$

Thus, $\text{Self}_C(\eta) \subseteq R_C(\eta)$.

Combining the two inclusions gives

$$R_C(\eta) = \text{Self}_C(\eta), \quad \text{ref. equation 2.}$$

The equality proves relative completeness with respect to the self-support target. $\square$

D.5 Termination

Theorem 17 (Termination). *For fixed bounds (K, B, T, R) , Algorithm 1 terminates.*

PROOF. We bound each phase.

Candidate enumeration. For each $v \in Y_C$, the algorithm maintains a finite set $D[v]$ and enforces $|D[v]| \leq B$. Hence, the total number of successful insertions into all candidate domains is bounded by

$$\sum_{v \in Y_C} |D[v]| \leq B|Y_C|.$$

The saturation test stops the enumeration once a full pass adds no new value or all domains reach their cap. Therefore, the enumeration loop is finite.

Candidate construction. After enumeration, each $D[v]$ is finite. Thus,

$$|\text{Cand}_C(\eta)| \leq \prod_{v \in Y_C} |D[v]| \leq B^{|Y_C|}.$$

The bound establishes finiteness of the candidate set.

Validation and repair. The algorithm iterates over the finite set $\text{Cand}_C(\eta)$. For each candidate and each $g \in C$, it performs at most R validation repetitions. Repair attempts are bounded by T , and each repair invokes only finitely many calls to **Propose** and **Validate**, bounded by K and R .

All phases are finite; thus Algorithm 1 terminates. $\square$

E Complexity

We prove Theorem 7 under bounded generation, finite candidate coverage, and exactness from Sections 3 and 6, together with the oracle computability assumption in Section 7. The proof uses the SCC-condensation result for acyclic and stratified evaluation. For well-founded evaluation, we bound the alternating construction of Definition 21 directly.

E.1 Decision problem and computational model

An instance consists of a pattern $\mathcal{P}$, an RDF graph $\mathcal{G}$, a typed mapping μ , and an applicable semantics $\text{Sem} \in \{\text{acyc}, \text{strat}, \text{wfs}\}$. The question is whether μ belongs to the selected answer relation; for $\text{Sem} = \text{wfs}$, membership refers to the true component. Let

$$n := |\mathcal{P}| + |\mathcal{G}| + |\mu|, \quad V := \text{var}(\mathcal{P}) \quad \text{and} \quad r := |V|,$$

where $|\cdot|$ denotes encoding length. Data complexity fixes $\mathcal{P}$ and Sem ; combined complexity allows both the pattern and the data to vary. The bounds n_1, n_2, K, B, T, R remain fixed.

The bound ℓ on generated-term representation length is a fixed constant over a fixed finite encoding alphabet Σ . Model, proposal, parsing, validation, lexicalization, compatibility, and representative operations are fixed deterministic unit-cost interfaces. We represent a generation request by the operator and typed placeholder binding; the symbolic answer consists of parsed tuples. Internal prompt expansion and raw model text belong to the external interface. We charge the evaluation procedure for manipulating polynomial-size symbolic inputs and outputs. Let $p(n)$ be a uniform polynomial bound on the space and symbolic work required for one mapping operation, filter test, or fully instantiated validation.

Oracle classes and reductions. Let $\mathcal{O}$ be the fixed family of interfaces. Requests and responses occupy the evaluator's charged storage. The upper-bound classes are $\text{PTIME}^{\mathcal{O}}$, $\text{PSPACE}^{\mathcal{O}}$, and $\text{EXPTIME}^{\mathcal{O}}$. We use ordinary polynomial-time many-one reductions, with no oracle calls.

The oracle computability assumption requires one deterministic procedure and fixed polynomials s and b such that, on an encoded symbolic request z (including operation and model identifiers), the procedure halts with the interface response, using at most $s(|z|)$ space and producing at most $b(|z|)$ output symbols. The procedure itself makes no external oracle calls. Uniformity requires the same polynomial bounds for all legal requests and model specifiers. The assumption concerns interface computability; we retain unit-cost accounting for the evaluator.

Lemma 11. $\text{PSPACE}^{\mathcal{O}} = \text{PSPACE}$.

PROOF OF LEMMA 11. Let $L \in \text{PSPACE}^{\mathcal{O}}$ be decided by an oracle machine M using polynomial space $q_M(n)$ on inputs of length n . The charged space includes request and response storage; therefore, each request has length at most $q_M(n)$.

We simulate M by an ordinary deterministic machine. At an oracle instruction, we retain the current configuration of M and run the uniform interface procedure on the request. The procedure uses at most $s(q_M(n))$ workspace cells, and a separate response buffer needs at most $b(q_M(n))$ cells. We copy the response to the simulated machine, release the temporary workspace, and resume

M . Thus, the total space is at most

$$O(q_M(n) + s(q_M(n)) + b(q_M(n))) = n^{O(1)}.$$

The procedure and M halt. Sequential oracle calls reuse the same workspace, regardless of their number. Therefore, $L \in \text{PSPACE}$. The reverse inclusion follows by ignoring the interfaces. $\square$

The lemma does not assert $\text{PTIME}^{\mathcal{O}} = \text{PTIME}$. We interpret data-time bounds in the stipulated unit-cost model. In particular, the additional assumption permits interface implementations whose running time is exponential in request length.

E.2 Finite mapping universe

For $v \in V$, let E_v be the equivalence relation induced by the reflexive and transitive closure of $\approx_v$. The representative rep_v is selected using the common order $\preceq$ of Section 3.

Lemma 12. For typed terms a and variables $v, w \in V$,

$$\text{rep}_v(\text{rep}_w(a)) = \begin{cases} \text{rep}_v(a), & \theta(v) \leq \theta(w), \\ \text{rep}_w(a), & \theta(v) > \theta(w). \end{cases}$$

For every finite set A_0 of typed terms, the set

$$A := A_0 \cup \{\text{rep}_v(a) \mid a \in A_0, v \in V\} \quad (3)$$

satisfies $\text{rep}_v(A) \subseteq A$ for all $v \in V$ and $|A| \leq (r+1)|A_0|$.

PROOF OF LEMMA 12. If $\theta(v) \leq \theta(w)$, then $E_w \subseteq E_v$. Therefore, $a E_w \text{rep}_w(a) \implies a E_v \text{rep}_w(a) \implies \text{rep}_v(\text{rep}_w(a)) = \text{rep}_v(a)$. If $\theta(v) > \theta(w)$, let $b := \text{rep}_w(a)$. Then,

$$b \in [b]_{E_v} \subseteq [a]_{E_w} \quad \text{and} \quad b = \min_{\preceq} [a]_{E_w}.$$

Thus, $b = \min_{\preceq} [b]_{E_v}$, and $\text{rep}_v(b) = b$. Both cases map each term of (3) back into A . Finally,

$$|A| \leq |A_0| + \sum_{v \in V} |\text{rep}_v(A_0)| \leq (r+1)|A_0|.$$

$\square$

Let Δ be the fixed set of typed generated terms permitted by the representation bound, and let

$$d_\ell := \sum_{i=0}^{\ell} |\Sigma|^i.$$

Term encodings determine typed values; therefore, $|\Delta| \leq d_\ell$. Let A_0 contain Δ and the RDF-typed terms occurring in $\mathcal{G}$ or as constants in $\mathcal{P}$. We construct A by (3), and let

$$\mathcal{U} := \{\eta : V \rightarrow A\} \quad \text{and} \quad N := |\mathcal{U}| = (|A| + 1)^r. \quad (4)$$

The additional choice for each variable represents an unbound value.

Lemma 13. The universe $\mathcal{U}$ is closed under one-step evaluation, reduct evaluation, and SCC macro-operators with contexts in $\mathcal{U}$. The bounds are

$$|A| \leq (r+1)(3|\mathcal{G}| + |\mathcal{P}| + d_\ell), \quad N \leq 2^{O(n \log(n+2))}. \quad (5)$$

For fixed $\mathcal{P}$, N is polynomial in $|\mathcal{G}|$. Enumeration of $\mathcal{U}$ takes $Nn^{O(1)}$ time and $n^{O(1)}$ space.

PROOF OF LEMMA 13. We prove closure by induction on the evaluation clauses.

BGP. Every binding has RDF type and a value occurring in $\mathcal{G}$, hence belongs to A_0 .

GenOp. The context values belong to A , and every generated value belongs to $\Delta \subseteq A_0$. Shared bindings are replaced only by their canonical representatives. Lemma 12 therefore keeps all bindings in A .

Algebraic constructors. Canonical union preserves A by Lemma 12. Projection, filtering, union, and the unmatched branches of difference, minus, and optional introduce no terms. The induction step follows. The argument also applies when a negative operand is evaluated against a fixed $J \subseteq \mathcal{U}$, proving reduct closure.

SCC macro-operators. Every candidate is $\eta \uplus v$, where $\eta \in \mathcal{U}$ and v has generated values. Canonical union, validation, restriction to Y_C , and union with the outer binding preserve A by the BGP, GenOp, and algebraic cases.

There are at most $3|\mathcal{G}|$ graph terms and at most $|\mathcal{P}|$ query constants. Thus, Lemma 12 gives the first bound in (5), and

$$\log N = r \log(|A| + 1) = O(n \log(n + 2)).$$

If $\mathcal{P}$ is fixed, both r and d_r are constants, and $N = O((|\mathcal{G}| + 2)^r)$, with a constant depending on $\mathcal{P}$. We compute A by representative calls and deduplication. Enumeration of $\mathcal{U}$ uses an r -digit counter in base $|A| + 1$ and stores one mapping at a time. The space requirement is

$$O(r \log(|A| + 1)) + p(n) = n^{O(1)},$$

and the total time is $Nn^{O(1)}$. $\square$

For finite grounding, we use

$$\Omega_{\mathcal{P}} = \text{Cand}(\mathcal{P}) = \mathcal{U}.$$

Lemma 13 establishes the sound over-approximation required in Section 4.5 and ensures that no one-step or reduct output is discarded. Well-founded evaluation is relative to the specified carrier. The local sets $\text{Cand}_C(\eta)$ of Section 6 remain proposal-generated sets. If $\mu \notin \mathcal{U}$, membership is rejected for all three evaluation regimes.

E.3 Local candidate bounds

For an SCC C and an outer binding η , we write $D_C^\eta[v]$ for the domains obtained by Algorithm 1. Let

$$y_C := |Y_C|, \quad m_C := |C| \quad \text{and} \quad N_C := B^{y_C}.$$

We use $B^0 = 1$, including when $B = 0$. A complete proposal pass is saturated if it inserts no new value. We use the deterministic proposal policy of the executable interface.

Lemma 14. *For fixed C and η , the proposal phase makes at most $By_C + 1$ complete passes and uses polynomial symbolic time and space in combined complexity.*

PROOF OF LEMMA 14. Each insertion increases $\sum_{v \in Y_C} |D_C^\eta[v]|$, and

$$\sum_{v \in Y_C} |D_C^\eta[v]| \leq By_C.$$

At most By_C passes insert a value, and at most one further pass detects saturation. The total is therefore at most $By_C + 1$. Each pass makes m_C proposal calls, each returning at most K bindings with at

most y_C coordinates. Copying context values and sentinel tokens, inspecting proposals, and updating domains take at most

$$(By_C + 1)(m_C + 1)(K + 1)(y_C + 1)p(n) = n^{O(1)}$$

symbolic time. The domains contain at most By_C values, giving the same polynomial space bound. $\square$

Lemma 15. *For every C and outer binding η ,*

$$|\text{Cand}_C(\eta)| \leq N_C \quad \text{and} \quad |R_C(\eta)| \leq N_C.$$

For every context supply S of outer bindings for C ,

$$|\llbracket \text{GenSCC}_C \rrbracket_S^\mathcal{G}| \leq N_C |S|.$$

PROOF OF LEMMA 15. The candidate definition and $|D_C^\eta[v]| \leq B$ give

$$\begin{aligned} |\text{Cand}_C(\eta)| &\leq \prod_{v \in Y_C} |D_C^\eta[v]| \\ &\leq B^{y_C} = N_C. \end{aligned}$$

Validation selects candidates and projection identifies some of them. Hence, $|R_C(\eta)| \leq |\text{Cand}_C(\eta)|$. By the macro-operator definition,

$$\begin{aligned} |\llbracket \text{GenSCC}_C \rrbracket_S^\mathcal{G}| &\leq \sum_{\substack{\eta \in S \\ X_C \subseteq \text{dom}_\eta}} |R_C(\eta)| \\ &\leq N_C |S|. \end{aligned}$$

Lemma 16. *Given an SCC C , an outer binding $\eta \in \mathcal{U}$, and $\rho \in \mathcal{U}$ with $\text{dom}_\rho = Y_C$, membership in $R_C(\eta)$ is decidable in polynomial symbolic time and space.*

PROOF OF LEMMA 16. We test the candidate determined by η and ρ . Since $\text{dom}_\eta \cap Y_C = \emptyset$, the only candidate with Y_C -projection ρ is $\eta \cup \rho$. Thus,

$$\begin{aligned} \rho \in R_C(\eta) &\iff \forall v \in Y_C : \rho(v) \in D_C^\eta[v] \\ &\quad \text{and} \quad \text{SS}_C(\eta \cup \rho). \end{aligned} \tag{6}$$

An accepted projection satisfies both conditions by the candidate and self-support definitions. Conversely, both conditions make $\eta \cup \rho$ an accepted candidate with projection ρ .

Lemma 14 constructs the domains in polynomial time. We test each coordinate against at most B values. The candidate binds every variable in $X_C \cup Y_C$; each self-support conjunct is one fully instantiated validation. There are m_C conjuncts, each requiring polynomial symbolic work. The entire test therefore uses polynomial time and space. If $Y_C = \emptyset$, the unique projection is empty, and the same equivalence applies. $\square$

E.4 Acyclic and stratified upper bounds

We use the exact SCC condensation of Section 6.4. Let m be the number of syntax occurrences of $\mathcal{P}$, and let $C_1, \dots, C_q$ be the components in topological order, where $q \leq m$. A recursive call retains the current ambient pattern, the designated occurrence, the omitted original occurrences, and the largest available component index. Supplies and component results are represented by references. Let

$$D_m := (m + 1)^3, \quad H_m := (2m + 4)^{D_m + 1}.$$

Lemma 17. *Recursive evaluation of the exact condensation has depth at most D_m and polynomial-size call descriptions. The same bounds hold for one-step reduct evaluation with fixed supplies S, J .*

PROOF OF LEMMA 17. We count the possible recursive calls along one path. An algebraic call descends to a strict subpattern. A surrounding-pattern call returns to the ambient pattern after omitting the designated occurrence; subsequent calls retain all omissions. Between successive omissions, algebraic descent has at most $m + 1$ levels. There are at most m omissions, giving at most $(m + 1)^2$ calls while the available component index remains fixed.

By Section 6.4, the input of C_i uses only components C_j with $j < i$. Computing that input therefore lowers the available index to $i - 1$. The local test for $R_{C_i}(\eta)$ makes no recursive query call by Lemma 16. There are at most q index decreases. Hence, the full path has at most

$$(q + 1)(m + 1)^2 \leq D_m$$

calls. Context selection and the two OPTIONAL branches use the current frame; recursive calls concern the original operands. For a reduct, negative operands use the prescribed fixed J -evaluation. Switching context mode retains the current pattern and all omissions, and the same omission and descent count applies.

We store the pruned syntax with original occurrence identifiers, an m -bit omission vector, and component indices and variable lists. The pruning rules of Definition 2 only remove syntax or insert the empty pattern. Every description therefore has polynomial size. Finite local loops and the path bound prove termination. $\square$

For a call u , let Ω_u be the specified relation, and let $\text{MEM}(u, \alpha)$ be the Boolean membership test defined in the next proof. The root call u_* evaluates the exact condensation. Thus, Section 6.4 identifies Ω_{u_*} with $\llbracket \mathcal{P} \rrbracket_{\mathcal{G}}^{\text{Sem}}$ for $\text{Sem} \in \{\text{acyc}, \text{strat}\}$.

Lemma 18. *Acyclic and stratified evaluation-membership belong to PSPACE^O.*

PROOF OF LEMMA 18. We test one mapping at a time and recompute intermediate answers when needed. All quantified mappings range over $\mathcal{U}$ and are enumerated sequentially by Lemma 13. We prove $\text{MEM}(u, \alpha) \iff \alpha \in \Omega_u$ by induction on the remaining recursion depth.

Atomic patterns. For a BGP Q , we accept exactly when α is RDF-typed, $\text{dom}_\alpha = \text{var}(Q)$, and $\alpha(Q) \subseteq \mathcal{G}$. The empty pattern accepts exactly $\emptyset$; a disabled occurrence rejects every mapping. The defining clauses establish the base cases.

Algebraic operators. Assume exact membership tests for the operand calls v, v_1, v_2 . For projection onto W , filter F , and union, respectively, we use

$$\text{MEM}(u, \alpha) \iff \exists \beta : \beta|_W = \alpha \wedge \text{MEM}(v, \beta),$$

$$\text{MEM}(u, \alpha) \iff F^\alpha = \top \wedge \text{MEM}(v, \alpha),$$

$$\text{MEM}(u, \alpha) \iff \text{MEM}(v_1, \alpha) \vee \text{MEM}(v_2, \alpha).$$

Join membership holds exactly when some β, γ satisfy

$$\text{MEM}(v_1, \beta) \wedge \text{MEM}(v_2, \gamma) \wedge \beta \sim \gamma \wedge \alpha = \beta \uplus \gamma.$$

For difference, we require $\text{MEM}(v_1, \alpha)$ and the absence of a mapping γ satisfying

$$\text{MEM}(v_2, \gamma) \wedge \alpha \sim \gamma \wedge F^{\alpha \uplus \gamma} = \top.$$

For minus, we replace the final conjunct by $\text{dom}_\alpha \cap \text{dom}_\gamma \neq \emptyset$. We form canonical unions only after compatibility succeeds. A filter error satisfies neither a filter test nor a blocking test. For OPTIONAL, we test the defining filtered-join and difference branches sequentially within the current frame.

The induction hypothesis identifies each enumerated witness with a mapping in the required operand relation. The projection and join clauses require exactly the stated witnesses; filter and union use the stated Boolean conditions. Exhaustive enumeration proves absence of a blocking witness for difference and minus. OPTIONAL combines those established cases. Each test therefore proves both directions of the membership equivalence.

Contexts and components. A surrounding-context call evaluates the prescribed pruned pattern and retains exactly the mappings binding the required placeholders. The induction hypothesis gives the pruned relation, and the domain test gives the required restriction. A base-mode context accepts exactly the empty mapping.

Let v_0 supply the incoming outer bindings of C . By the macro-operator clause,

$$\begin{aligned} \text{MEM}(u, \alpha) \iff \exists \eta, \rho : & \text{MEM}(v_0, \eta) \\ & \wedge X_C \subseteq \text{dom}_\eta \\ & \wedge \text{dom}_\eta \cap Y_C = \emptyset \\ & \wedge \text{dom}_\rho = Y_C \wedge \rho \in R_C(\eta) \\ & \wedge \alpha = \eta \cup \rho. \end{aligned}$$

The domain conditions are those of an outer binding in Section 6.1. We enumerate η, ρ and use Lemma 16 for the local test. The induction hypothesis gives the incoming relation, and component exactness gives precisely the declarative output projections. The equivalence proves both directions for component calls and completes the induction. At the root, the test therefore decides the required semantic membership.

Space. One frame stores the call description, a constant number of mappings and counters, and local validation data. Lemmas 13 and 16 bound the space per frame by one polynomial $q_0(n)$. Sequential recursive calls reuse their workspace. By Lemma 17, total space is at most

$$D_m q_0(n) = n^{O(1)}.$$

The depth bound and finite enumerations prove termination, including the negative tests. Hence, membership belongs to PSPACE^O. $\square$

Lemma 19. *Acyclic and stratified evaluation-membership have polynomial data complexity in the unit-cost interface model.*

PROOF OF LEMMA 19. We bound the time of the membership algorithm for fixed $\mathcal{P}$. Each call enumerates at most N^2 pairs of mappings and makes at most a constant number of recursive tests per pair. Local tests take polynomial time by Lemma 16. Thus, branching is $O((N + 1)^2)$ and local work is $(N + 1)^2 n^{O(1)}$. The call depth is at most D_m , giving total time

$$(N + 1)^{O(D_m)} n^{O(1)}.$$

For fixed $\mathcal{P}$, D_m is constant and N is polynomial in the data size by Lemma 13. The time bound is therefore polynomial. Correctness follows from Lemma 18. $\square$

E.5 Well-founded upper bounds

For $J \subseteq \mathcal{U}$, let

$$\Gamma_{\mathcal{P}}(J) := \text{lfp}(S \mapsto \Phi_{\mathcal{P}}^J(S)).$$

The reduct is monotone in S , and $\Gamma_{\mathcal{P}}$ is antitone in J by Theorem 13.

Lemma 20. *Given explicit $S, J \subseteq \mathcal{U}$, one reduct evaluation takes at most $H_m(N+1)^3 p(n)$ time in the unit-cost interface model, after increasing the fixed polynomial p if necessary.*

PROOF OF LEMMA 20. We evaluate the reduct recursively using explicit finite relations. Lemma 17 bounds the depth by D_m . An algebraic operator has at most two operands; OPTIONAL evaluates those operands for the filtered-join and difference branches. A generative occurrence also evaluates the prescribed surrounding pattern. Computing child relations before scanning mappings gives at most $2m+4$ recursive calls per node, hence fewer than H_m nodes in the call tree.

Each relation has at most N mappings by Lemma 13. A BGP enumerates $\mathcal{U}$ and tests the ground triples. A generator inspects at most N contexts and at most $n_1 n_2$ parsed tuples per context. Projection, filtering, and union scan at most $2N$ rows; join, difference, and minus inspect at most N^2 pairs. OPTIONAL uses the filtered-join and difference clauses. Negative operands use the prescribed fixed J -evaluation, including their surrounding contexts. Each clause therefore returns the specified relation once the recursive operands are correct; induction on the finite call tree proves correctness.

Locating and deduplicating an output mapping takes at most $Np(n)$ time by a linear scan. Including symbolic manipulation and context construction, the work per node is at most $(N+1)^3 p(n)$. Multiplication by the node bound gives $H_m(N+1)^3 p(n)$. $\square$

Lemma 21. *Well-founded evaluation-membership has polynomial data complexity and an EXPTIME^O combined upper bound.*

PROOF OF LEMMA 21. We count the inner least-fixpoint iterations and the outer alternating stages. For fixed J , let $S_0 := \emptyset$ and $S_{i+1} := \Phi_{\mathcal{P}}^J(S_i)$. The base inclusion is $S_0 \subseteq S_1$. If $S_i \subseteq S_{i+1}$, monotonicity gives

$$S_{i+1} = \Phi_{\mathcal{P}}^J(S_i) \subseteq \Phi_{\mathcal{P}}^J(S_{i+1}) = S_{i+2}.$$

The sequence is therefore ascending and stabilizes within $N+1$ evaluations. For any fixpoint F , induction gives $S_i \subseteq F$: the base is empty, and $S_{i+1} = \Phi_{\mathcal{P}}^J(S_i) \subseteq \Phi_{\mathcal{P}}^J(F) = F$. The stable value is therefore $\Gamma_{\mathcal{P}}(J)$. Lemma 20 gives

$$T_{\Gamma}(n, N) \leq H_m(N+1)^4 p(n). \quad (7)$$

Definition 21 has the lower and upper form

$$\begin{aligned} L_0 &:= \emptyset, & U_0 &:= \mathcal{U}, \\ L_{i+1} &:= \Gamma_{\mathcal{P}}(U_i), & U_{i+1} &:= \Gamma_{\mathcal{P}}(L_{i+1}). \end{aligned}$$

By Theorem 13,

$$L_i \subseteq L_{i+1} \subseteq U_{i+1} \subseteq U_i.$$

Each changing stage adds a member to L or removes one from U . At most $2N$ stages change the pair, and one further stage detects equality. Each stage computes two values of $\Gamma_{\mathcal{P}}$. Subset comparisons and copies take $O(N+1)$ time each and are absorbed by increasing p . Hence,

$$T_{\text{wfs}}(n, N) \leq 2(2N+1)H_m(N+1)^4 p(n). \quad (8)$$

The stable lower approximation is the true component by Definition 21 and Theorem 13.

For fixed $\mathcal{P}$, H_m is constant and N is polynomial in $|\mathcal{G}|$, giving polynomial data time. In combined complexity,

$$\log H_m = O(m^3 \log(m+2)) \quad \text{and} \quad \log(N+1) = O(n \log(n+2)).$$

Equation (8) therefore gives $T_{\text{wfs}} \leq 2^{n^{O(1)}}$. The final membership test costs at most $Np(n)$ and is absorbed in the bound. $\square$

E.6 Combined lower bounds

For an ordinary SPARQL mapping α , we define the RDF-typed lift by

$$\text{lift}(\alpha) := \{v \mapsto (\alpha(v), \text{rdf}) \mid v \in \text{dom}_{\alpha}\}.$$

Evaluation of standard SPARQL expressions built from triple patterns and unconditional OPT is PSPACE-complete [36, Theorem 4].

We define the translation $\tau_0(Q)$ and an anchor a_Q by induction. For a nonground triple pattern t , let $\tau_0(t) := \{t\}$ and choose $a_t \in \text{var}(t)$. For a ground triple $t = (s, p, o)$, choose a fresh variable z_t , distinct for each occurrence, and let

$$\tau_0(t) := \text{Filter}_{z_t=o}(\{(s, p, z_t)\}) \quad \text{and} \quad a_t := z_t.$$

For $Q = Q_1 \text{ OPT } Q_2$, let

$$a_Q := a_{Q_1} \quad \text{and} \quad \tau_0(Q) := \tau_0(Q_1) \text{ Opt}_{\text{bound}(a_Q)} \tau_0(Q_2).$$

Finally, let $\tau(Q) := \text{Proj}_{\text{var}(Q)}(\tau_0(Q))$.

Lemma 22. *The translation is polynomial-time and yields a well-formed, GenOp-free pattern satisfying*

$$\llbracket \tau(Q) \rrbracket_{\mathcal{G}} = \{\text{lift}(\alpha) \mid \alpha \in \llbracket Q \rrbracket_{\mathcal{G}}^{\text{SPARQL}}\}. \quad (9)$$

PROOF OF LEMMA 22. We use GenOp-free conservativity from Section 3. For each source subexpression Q , let $T_Q := \llbracket \tau_0(Q) \rrbracket_{\mathcal{G}}$ and $S_Q := \llbracket Q \rrbracket_{\mathcal{G}}^{\text{SPARQL}}$. We prove

$$\begin{aligned} \forall \beta \in T_Q : a_Q \in \text{dom}_{\beta}, \quad \text{and} \\ \{\beta|_{\text{var}(Q)} \mid \beta \in T_Q\} &= \{\text{lift}(\alpha) \mid \alpha \in S_Q\}. \end{aligned}$$

Triple patterns. The nonground case follows from BGP evaluation. For a ground $t = (s, p, o)$, the target produces exactly $\{z_t \mapsto (o, \text{rdf})\}$ when $t \in \mathcal{G}$, and no mapping otherwise. Projection onto the empty source-variable set gives the standard value of the ground triple.

OPTIONAL. Assume the two claims for Q_1, Q_2 . Every left result binds a_Q ; therefore, $\text{bound}(a_Q)$ is true on each compatible extension. Fresh variables of the two children are disjoint from each other and from every source variable. Therefore, for $\beta_i \in T_{Q_i}$,

$$\beta_1 \sim \beta_2 \iff \beta_1|_{\text{var}(Q_1)} \sim \beta_2|_{\text{var}(Q_2)}.$$

Compatible joins project to standard unions. The equality in the induction hypothesis also gives

$$\exists \beta_2 \in T_{Q_2} : \beta_1 \sim \beta_2 \iff \exists \alpha_2 \in S_{Q_2} : \alpha_1 \sim \alpha_2,$$

where $\text{lift}(\alpha_1) = \beta_1|_{\text{var}(Q_1)}$. Thus, the difference branch retains precisely the representatives of unmatched source mappings. The anchor remains bound in both branches, completing the induction.

The root projection proves (9). All filter variables occur in their operands, and the final projection uses source variables. Hence, the target is well formed. There are a bounded number of new

syntax nodes per source node and at most logarithmic overhead for fresh variable indices. The translation is polynomial-time and polynomial-size. $\square$

Lemma 23. *Evaluation-membership is PSPACE-hard in combined complexity for each semantics in Theorem 7.*

PROOF OF LEMMA 23. We map a standard OPTIONAL instance $(Q, \mathcal{G}, \alpha)$ to

$$(\tau(Q), \mathcal{G}, \text{lift}(\alpha), \text{Sem}).$$

If $\text{dom}_\alpha \not\subseteq \text{var}(Q)$, we use a fixed negative instance consisting of a nonempty BGP, the empty graph, and the empty mapping. Otherwise, (9) preserves membership.

Let $A_Q := \llbracket \tau(Q) \rrbracket_{\mathcal{G}}$. The target is GenOp-free; the dependency graph is empty, and the consequence operator is the constant map $S \mapsto A_Q$. Both acyclic and stratified evaluation therefore return A_Q .

The target contains no negated filter. Every negative OPTIONAL operand is also GenOp-free and hence independent of the context. Induction on the target syntax gives

$$\Phi_{\tau(Q)}^J(S) = A_Q \quad \text{for every } S, J \subseteq \mathcal{U}.$$

Therefore, $\Gamma_{\tau(Q)}(J) = A_Q$ for every J , and the first alternating stage yields $L_1 = U_1 = A_Q$. Well-founded membership has the same value.

All three reductions are polynomial-time and preserve yes- and no-instances. Since the target contains no generative SCC, the local coverage and exactness conditions are vacuous. $\square$

E.7 Proof of Theorem 7

PROOF OF THEOREM 7. We combine the upper bounds with the GenOp-free reduction. Lemmas 19 and 21 give polynomial data time for all three semantics in the stipulated unit-cost model.

For acyclic and stratified evaluation, Lemma 18 gives a PSPACE^O upper bound. Lemma 11 gives $\text{PSPACE}^O = \text{PSPACE}$, and Lemma 23 gives the matching lower bound. Both evaluation problems are therefore PSPACE-complete.

For well-founded evaluation, Lemma 21 gives $2^{n^{O(1)}}$ symbolic time. Each interface request contains an operator and polynomial-size bindings, terms, or candidate domains. The uniform interface procedure therefore uses polynomial space and produces polynomial output. A halting deterministic computation with those bounds has at most $2^{n^{O(1)}}$ configurations and cannot repeat a configuration. Thus, each response takes at most $2^{n^{O(1)}}$ ordinary time. Simulating the at most exponentially many interface calls preserves the EXPTIME upper bound. Lemma 23 gives PSPACE-hardness. $\square$